

A PROJECT REPORT ON

AI POWERED RESUME SCREENING SYSTEM

Using Natural Language Processing, Machine Learning and Computer Vision (OCR)

Submitted in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

Course Code: MCSP-232

Submitted by

Student Name

Enrolment No.: XXXXXXXXXX

Under the guidance of

Guide Name, Designation

SCHOOL OF COMPUTER AND INFORMATION SCIENCES
INDIRA GANDHI NATIONAL OPEN UNIVERSITY

Maidan Garhi, New Delhi – 110068

Study Centre: Study Centre Name & Code · Regional Centre: Regional Centre

Session: 2026

Certificate of Originality

This is to certify that the project report entitled “**AI Powered Resume Screening System**” submitted to Indira Gandhi National Open University in partial fulfilment of the requirement for the award of the degree of **Master of Computer Applications (MCA)** is an original work carried out by Mr./Ms. **Student Name**, Enrolment No. **XXXXXXXXXX**, under my guidance.

The matter embodied in this project is authentic and is genuine work done by the student and has not been submitted, whether to this University or to any other University / Institute, for the fulfilment of the requirement of any course of study.

Signature of the Student

Name: **Student Name**

Enrolment No.: **XXXXXXXXXX**

Date: **DD/MM/YYYY**

Place: **City**

Signature of the Guide

Name: **Guide Name**

Designation: **Designation**

Address: **Guide Address**

Date: **DD/MM/YYYY**

Declaration

I hereby declare that the project work entitled “**AI Powered Resume Screening System**” submitted to Indira Gandhi National Open University, New Delhi, is a record of original work done by me under the guidance of **Guide Name**. This project work is submitted in partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications (MCA)**.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma. All sources of information used in this report have been duly acknowledged.

Place: **City**

Student Name

Date: **DD/MM/YYYY**

Enrolment No.: **XXXXXXXXXX**

Acknowledgement

I would like to express my sincere gratitude to my project guide, **Guide Name**, for the valuable guidance, constant encouragement and constructive suggestions provided throughout the course of this project.

I am thankful to the School of Computer and Information Sciences, Indira Gandhi National Open University, and to the coordinator and staff of my study centre, **Study Centre Name**, for providing the academic framework and support that made this work possible.

I also acknowledge the open-source communities behind Python, Streamlit, scikit-learn, spaCy, NLTK, OpenCV, Tesseract OCR, PyMuPDF and SQLite, whose freely available tools form the foundation of this system.

Finally, I thank my family and friends for their patience, motivation and support during the development of this project.

Student Name
Enrolment No.: **XXXXXXXXXX**

Abstract

Recruiters routinely receive hundreds of resumes for a single opening, and manual screening is slow, inconsistent and prone to bias. Many applicant tracking systems also fail on scanned or image-based resumes, silently discarding qualified candidates. This project presents the design and implementation of an **AI Powered Resume Screening System** that automates resume evaluation by combining Natural Language Processing (NLP), Machine Learning (ML) and Computer Vision (CV).

The system accepts resumes in PDF, DOCX, TXT and image formats. Documents with a text layer are parsed directly, while scanned documents pass through an OpenCV pipeline — grayscale conversion, non-local-means denoising, projection-profile deskewing and Otsu binarisation — before text is recognised with the Tesseract OCR engine. The extracted text is cleaned, tokenised, filtered for stopwords and lemmatised with spaCy, and structured information (name, e-mail, phone, skills from a vocabulary of 119 canonical skills, years of experience, education and resume sections) is extracted.

Candidate–job matching is performed with a TF-IDF vector space model and cosine similarity, combined with required-skill coverage and experience fit into a transparent weighted score (55 % content similarity, 35 % skills, 10 % experience). Every result records its component scores together with matched and missing skills, making each ranking explainable. The application is built with Python and Streamlit, stores data in a normalised SQLite database of nine tables, secures access with bcrypt-hashed passwords and role-based authorisation, and generates six management reports exportable as CSV or PDF.

On the evaluation dataset of seven resumes (five text-based, two scanned) and three job postings, the system extracted name, e-mail, phone, education and experience correctly for all seven resumes, achieved an average OCR word confidence of 94.2 %, and ranked the intended best-fit candidate first for every job posting. All 12 automated tests pass. The work demonstrates that classical, lightweight AI techniques can deliver an accurate, explainable and privacy-friendly screening tool that runs entirely on a local machine.

Keywords: resume screening, natural language processing, TF-IDF, cosine similarity, OCR, OpenCV, Tesseract, Streamlit, SQLite, explainable AI.

Table of Contents

Certificate of Originality	ii
Declaration	iii
Acknowledgement.....	iv
Abstract	v
Table of Contents	vi
List of Figures	ix
List of Tables.....	x
List of Abbreviations.....	xi
Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives.....	2
1.4 Scope of the Project.....	2
1.5 Project Category	2
1.6 Advantages of the Proposed System	3
1.7 Organisation of the Report	3
Chapter 2: System Analysis	4
2.1 Existing System.....	4
2.2 Proposed System	4
2.3 Feasibility Study.....	5
2.3.1 Technical Feasibility	5
2.3.2 Economic Feasibility	5
2.3.3 Operational Feasibility	5
2.3.4 Schedule Feasibility	5
2.4 Requirement Specification	6
2.4.1 Functional Requirements.....	6
2.4.2 Non-Functional Requirements	6
2.5 Hardware and Software Requirements.....	7
2.6 Technology Background	7
2.6.1 Natural Language Processing.....	7
2.6.2 TF-IDF and Cosine Similarity.....	8
2.6.3 Computer Vision and OCR	8
Chapter 3: System Design	9
3.1 System Architecture	9
3.2 Data Flow Diagrams.....	10

3.2.1 Level 0 — Context Diagram	10
3.2.2 Level 1 DFD	10
3.2.3 Level 2 DFD — NLP, OCR and ML	11
3.3 Entity Relationship Diagram	12
3.4 Use Case Diagram	13
3.5 Process Flowchart	14
3.6 Module Design	15
3.7 Database Design	16
3.8 User Interface Design	19
Chapter 4: Implementation	20
4.1 Technology Stack	20
4.2 Project Structure	20
4.3 Resume Upload and Parsing Module	21
4.4 NLP Processing Module	22
4.4.1 Text Preprocessing	22
4.4.2 Information Extraction	22
4.5 Computer Vision and OCR Module	23
4.6 Machine Learning Matching Module	24
4.7 Database Module	25
4.8 User Management and Authentication	25
4.9 Result and Reporting Module	26
Chapter 5: Testing	27
5.1 Testing Strategy	27
5.2 Automated Test Suite	27
5.3 Test Cases	28
5.4 Extraction Accuracy on the Evaluation Dataset	29
5.5 Defects Found and Fixed	29
Chapter 6: Results and Discussion	31
6.1 Application Screens	31
6.1.1 Login	31
6.1.2 Dashboard	31
6.1.3 Job Postings	32
6.1.4 Upload and OCR	33
6.1.5 Screening and Ranking	34
6.1.6 Candidates	35
6.1.7 Reports and Settings	36
6.2 Ranking Results	36

6.3 System Performance.....	38
6.4 Discussion	39
Chapter 7: Security Implementation	40
Chapter 8: Conclusion and Future Scope	41
8.1 Conclusion.....	41
8.2 Limitations	41
8.3 Future Scope.....	42
References	43
Appendix A: Installation and User Manual.....	44
A.1 Installation	44
A.2 Using the System.....	44
A.3 Configuration.....	45
Appendix B: Selected Source Code	46

List of Figures

Figure 1: Project schedule (Gantt chart)	5
Figure 2: Four-layer system architecture	9
Figure 3: DFD Level 0 (context diagram).....	10
Figure 4: DFD Level 1	11
Figure 5: DFD Level 2 — upload/OCR, NLP and ML matching.....	12
Figure 6: Entity Relationship diagram	13
Figure 7: Use case diagram	14
Figure 8: Resume processing flowchart	15
Figure 9: OpenCV preprocessing stages on a real scanned resume	24
Figure 10: Login screen.....	31
Figure 11: Dashboard — key metrics, top skills and score distribution	32
Figure 12: Dashboard — resume formats, status by job and recent uploads	32
Figure 13: Job Postings page.....	33
Figure 14: Upload Resumes — scanned resume processed with OCR.....	33
Figure 15: Screening & Ranking — threshold and ranked chart	34
Figure 16: Ranked candidates table with editable status.....	34
Figure 17: Explainable matching — component scores, matched and missing skills ...	35
Figure 18: Candidate profile with recommended job roles.....	35
Figure 19: Reports page — Matching Result Report with CSV/PDF export	36
Figure 20: Settings — password change and user management (admin)	36
Figure 21: Match scores of all candidates against all job postings	37

List of Tables

Table 1: Abbreviations used in this report	xi
Table 2: Functional requirements.....	6
Table 3: Non-functional requirements	6
Table 4: Hardware and software platform.....	7
Table 5: System modules	15
Table 6: Data dictionary — USER.....	16
Table 7: Data dictionary — CANDIDATE.....	16
Table 8: Data dictionary — SKILL.....	17
Table 9: Data dictionary — CANDIDATE_SKILL	17
Table 10: Data dictionary — RESUME.....	17
Table 11: Data dictionary — JOB_POSTING	17
Table 12: Data dictionary — JOB_SKILL	18
Table 13: Data dictionary — MATCHING_RESULT	18
Table 14: Data dictionary — ACTIVITY_LOG	18
Table 15: Technology stack and justification	20
Table 16: Source files and size.....	21
Table 17: Information extraction techniques	22
Table 18: Components of the match score	24
Table 19: Levels of testing performed	27
Table 20: Automated test results	27
Table 21: Functional test cases.....	28
Table 22: Extraction results per resume	29
Table 23: Defect log.....	29
Table 24: Ranking for “Machine Learning Engineer (NLP / CV)” (min. 2 yrs)	37
Table 25: Ranking for “Data Analyst” (min. 1 yrs)	37
Table 26: Ranking for “Full Stack Developer” (min. 3 yrs)	38
Table 27: System Performance Report (generated by the application).....	38
Table 28: Security mechanisms at various levels.....	40
Table 29: Configuration options (config.py).....	45

List of Abbreviations

Table 1: Abbreviations used in this report

Abbreviation	Full form
AI	Artificial Intelligence
ATS	Applicant Tracking System
CSV	Comma-Separated Values
CV	Computer Vision
DFD	Data Flow Diagram
DOCX	Office Open XML Word document
ER	Entity Relationship
FK / PK	Foreign Key / Primary Key
JD	Job Description
ML	Machine Learning
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
RDBMS	Relational Database Management System
SQL	Structured Query Language
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface

Chapter 1: Introduction

1.1 Background

Artificial Intelligence has changed the way organisations handle information-intensive tasks. Chatbots, recommendation engines and automated decision-support systems are now common in healthcare, education, finance and human resources. Recruitment is a particularly strong candidate for automation: a single job advertisement can attract hundreds of applications, and every resume has to be read, interpreted and compared against the job requirements before a shortlist can be prepared.

Resumes are *unstructured documents*. Two candidates with identical skills may describe them with different words (for example “ML” and “machine learning”), organise their resumes in different section orders and submit them in different file formats — including photographs or scans of printed resumes. Traditional keyword filters cannot cope with this variety, and manual screening does not scale.

This project applies three complementary branches of AI to the problem. **Natural Language Processing** converts free text into normalised, analysable tokens and extracts structured facts; **Machine Learning** techniques represent resumes and job descriptions as vectors and measure their similarity; and **Computer Vision** with **Optical Character Recognition** recovers text from scanned or image-based resumes so that no candidate is excluded because of file format.

1.2 Problem Statement

The recruitment process in many organisations involves manually screening a large number of resumes, which is time-consuming, inefficient and prone to human bias. Traditional methods often fail to match candidate profiles accurately with job requirements because they lack intelligent analysis of unstructured resume data. In addition, many systems are unable to process scanned or image-based resumes, leading to loss of important candidate information.

There is therefore a need for an automated and intelligent system that can efficiently extract, analyse and evaluate resume data — understanding textual content through NLP, performing accurate matching using ML, and handling diverse resume formats using CV and OCR — while keeping its decisions transparent to the recruiter.

1.3 Objectives

1. To develop an intelligent Resume Screening System using AI techniques.
2. To automate the extraction of useful information (contact details, skills, experience, education) from resumes using NLP.
3. To compare resumes with job descriptions using Machine Learning (TF-IDF vectorisation and cosine similarity).
4. To generate a matching score, rank candidates accordingly and explain each score.
5. To process both text-based and scanned resumes using OCR and Computer Vision.
6. To reduce manual effort and improve efficiency and consistency in the recruitment process.
7. To store and manage data efficiently in a relational database (SQLite).
8. To secure the application with hashed passwords, role-based access and validated file uploads.

1.4 Scope of the Project

The system covers the complete screening workflow within a local environment: recruiter authentication, creation of job postings with automatic skill detection, bulk upload of resumes in PDF, DOCX, TXT and common image formats, OCR for scanned documents, NLP-based information extraction, ML-based matching and ranking, automatic and manual shortlisting, and generation of six management reports. Data is persisted in SQLite and the user interface is a Streamlit web application served on localhost.

The current scope is a working prototype. Real-time integration with job portals, cloud deployment, very large databases and deep-learning language models (such as BERT) are outside the present scope and are discussed as future enhancements in Chapter 8.

1.5 Project Category

The project falls under the category of **Artificial Intelligence and Machine Learning**, and also involves **Natural Language Processing** and **Computer Vision**. Architecturally it is a data-driven web application with a relational database back end (RDBMS category).

1.6 Advantages of the Proposed System

- Automates screening and processes a batch of resumes in seconds rather than hours.
- Handles unstructured text and multiple formats, including scanned images, through OCR.
- Normalises skill synonyms (e.g. “sklearn” → scikit-learn, “NLP” → natural language processing) so that candidates are not penalised for wording.
- Produces objective, repeatable scores and shows matched and missing skills for every candidate (explainable matching).
- Keeps all data on the local machine, which is attractive for candidate privacy.
- Modular design that allows the matching model, skill vocabulary or database to be replaced independently.

1.7 Organisation of the Report

Chapter 2 analyses the existing system, feasibility and requirements. Chapter 3 presents the system design through architecture, data flow, ER, use-case and flowchart diagrams, module design and the data dictionary. Chapter 4 describes the implementation of each module with the key algorithms. Chapter 5 covers testing, including automated tests, test cases, extraction accuracy and defects found. Chapter 6 presents results and screenshots. Chapter 7 details the security mechanisms, and Chapter 8 concludes the report with limitations and future scope. Appendices contain the installation guide and user manual.

Chapter 2: System Analysis

System analysis studies the existing process, identifies its shortcomings and defines what the new system must do. The findings of this phase guided the design decisions described in Chapter 3.

2.1 Existing System

In most small and medium organisations resumes arrive by e-mail or through job portals and are screened manually by HR staff, sometimes with the help of basic keyword search in a spreadsheet or an Applicant Tracking System (ATS). The existing approach has the following limitations:

- **Time-consuming:** each resume must be opened and read; screening effort grows linearly with the number of applicants.
- **Inconsistent:** different reviewers, or the same reviewer at different times, apply criteria differently.
- **Keyword brittleness:** exact-match filters reject candidates who use synonyms or abbreviations.
- **Format blindness:** scanned or photographed resumes cannot be searched, so those candidates are often ignored.
- **No explanation:** keyword filters return pass/fail without showing why a candidate was ranked low.
- **Poor record keeping:** decisions and screening history are scattered across e-mails and spreadsheets.

2.2 Proposed System

The proposed AI Powered Resume Screening System addresses each limitation. Resumes of any supported format are uploaded in bulk; text is extracted directly or through OCR; NLP normalises and structures the content; and an ML matcher produces a 0–100 score per job posting that blends semantic content similarity, required-skill coverage and experience fit. Recruiters see a ranked, explainable list, can shortlist automatically by threshold or manually, and can export reports. All data and decisions are stored in a relational database with an activity log.

2.3 Feasibility Study

2.3.1 Technical Feasibility

All required technologies are mature, well documented and open source: Python 3, Streamlit for the UI, scikit-learn for TF-IDF and cosine similarity, spaCy and NLTK for NLP, OpenCV and Tesseract for OCR, PyMuPDF and python-docx for document parsing, and SQLite as an embedded database. The system runs on an ordinary laptop without a GPU. The project is therefore technically feasible.

2.3.2 Economic Feasibility

Every component is free and open source, and no paid cloud service or licence is required. Development needs only a standard computer. The cost of the system is limited to development effort, while the benefit — hours of recruiter time saved per opening — is immediate. The project is economically feasible.

2.3.3 Operational Feasibility

The interface runs in a web browser and follows a simple workflow (create job → upload resumes → review ranking → shortlist → export report). Recruiters need no knowledge of AI; scores are accompanied by plain-language explanations. The system is operationally feasible.

2.3.4 Schedule Feasibility

The work was planned over approximately eighteen weeks, as shown in the Gantt chart in Figure 1. The modular design allowed independent development and testing of each module within the academic schedule.

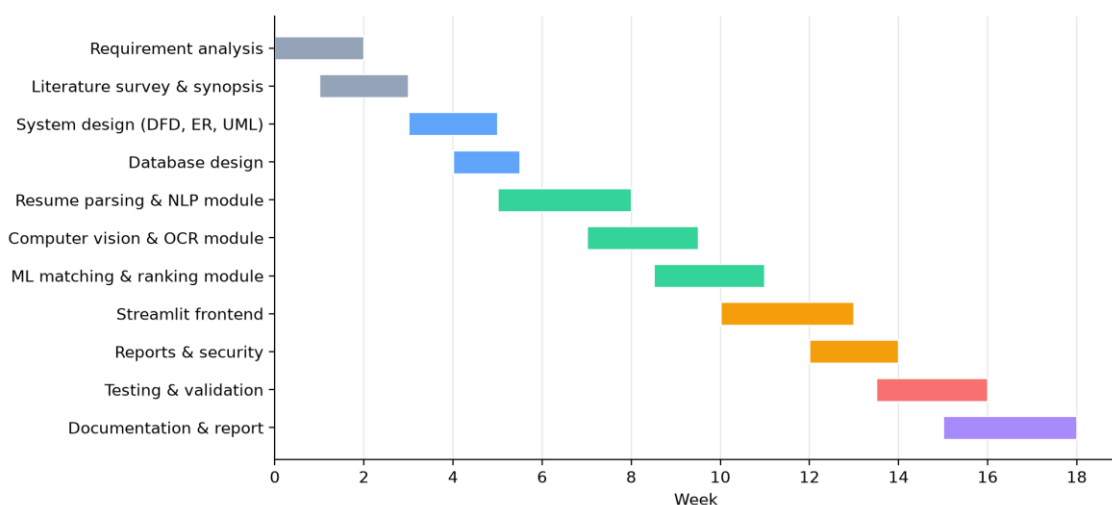


Figure 1: Project schedule (Gantt chart)

2.4 Requirement Specification

2.4.1 Functional Requirements

Table 2: Functional requirements

ID	Requirement	Module
FR-01	Users shall log in with e-mail and password; invalid credentials are rejected.	User Management
FR-02	Administrators shall create recruiter/admin accounts; users can change their password.	User Management
FR-03	Recruiters shall create job postings; required skills and minimum experience are detected automatically and can be edited.	Job Postings
FR-04	The system shall accept multiple resumes in PDF, DOCX, TXT, PNG, JPG, BMP and TIFF format.	Upload & Parsing
FR-05	The system shall validate file type, size (≤ 10 MB) and file signature before processing.	Upload & Parsing
FR-06	Scanned PDFs and images shall be processed with OpenCV preprocessing and Tesseract OCR.	CV & OCR
FR-07	The system shall extract name, e-mail, phone, skills, experience, education and sections.	NLP Processing
FR-08	The system shall compute a match score for every resume against a job and rank candidates.	ML Matching
FR-09	The system shall show matched and missing skills and component scores for each candidate.	ML Matching
FR-10	Recruiters shall shortlist candidates automatically by threshold or manually (Selected/Rejected).	Result & Reporting
FR-11	The system shall generate six reports and export them as CSV and PDF.	Result & Reporting
FR-12	All data and user actions shall be stored in the SQLite database.	Database

2.4.2 Non-Functional Requirements

Table 3: Non-functional requirements

Attribute	Requirement
Performance	A text resume should be parsed in under 1 second and a scanned page in under 10 seconds on an Intel Core i3 / 4 GB machine; matching should take milliseconds per resume.
Accuracy	Contact details and skills should be extracted reliably; the most suitable candidate should rank first.
Security	Passwords hashed with bcrypt; authenticated sessions; role-based access; validated uploads; parameterised SQL.
Usability	Browser-based UI with a clear workflow, progress feedback and explanations of scores.
Reliability	A malformed file must not crash the application; errors are reported per file.

Attribute	Requirement
Maintainability	Modular code (UI, core logic, data) with configuration in one file and an automated test suite.
Portability	Runs on Windows, Linux and macOS with Python 3.10+.
Privacy	All processing and storage happen locally; no resume data is sent to external services.

2.5 Hardware and Software Requirements

Table 4: Hardware and software platform

Category	Specification
Processor	Intel Core i3 or higher
RAM	Minimum 4 GB (8 GB recommended)
Hard disk	Minimum 100 GB (application and models need < 1 GB)
Operating system	Windows 10/11, Linux or macOS
Programming language	Python 3.10 or later (developed on Python 3.13)
Frontend	Streamlit 1.59, Plotly
Database	SQLite 3 (via Python sqlite3) with Pandas
NLP libraries	spaCy 3.8 (en_core_web_sm), NLTK 3.10
Machine learning	scikit-learn 1.9, NumPy
Computer vision & OCR	OpenCV 5 (headless), Tesseract OCR 5.4 via pytesseract
Document parsing	PyMuPDF (PDF), python-docx (DOCX)
Security	bcrypt
Reporting	ReportLab (PDF), Pandas (CSV)
Tools	VS Code, Git, pytest, Google Chrome / Microsoft Edge

2.6 Technology Background

2.6.1 Natural Language Processing

NLP converts human language into a form that programs can analyse. The pipeline used here consists of **tokenisation** (splitting text into words), **stopword removal** (discarding very frequent words such as “the” or “with” that carry no discriminating information) and **lemmatisation** (reducing inflected forms such as “developed”, “developing” and “develops” to the dictionary form “develop”). Rule-based information extraction with regular expressions and a curated skill vocabulary then identifies structured facts.

2.6.2 TF-IDF and Cosine Similarity

The **vector space model** represents each document as a vector with one dimension per term. TF-IDF weights a term highly when it is frequent in a document but rare across the collection. For term t in document d from a collection of N documents:

$$tf-idf(t, d) = (1 + \log tf(t, d)) \times (\ln[(1 + N) / (1 + df(t))]) + 1$$

where tf is the raw term count (sub-linear scaling) and df is the number of documents containing t . Vectors are L2-normalised and compared with **cosine similarity**, which measures the angle between two vectors and is independent of document length:

$$\cos(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

2.6.3 Computer Vision and OCR

OCR engines recognise characters most accurately on clean, level, high-contrast images. OpenCV is used to prepare scans: conversion to grayscale, **non-local-means denoising** to remove speckle, **deskewing** to make text lines horizontal, and **Otsu's method**, which automatically selects the global threshold that best separates ink from paper. Tesseract 5, an open-source LSTM-based OCR engine, then recognises the text and reports a confidence value for every word.

Chapter 3: System Design

System design translates the requirements into an architecture, data model and module structure. This chapter presents the design using standard structured-analysis and UML notation.

3.1 System Architecture

The system follows a four-layer architecture (Figure 2). The **presentation layer** is a Streamlit web interface. The **application layer** contains authentication, document parsing, database access and report generation. The **intelligence layer** holds the NLP, ML matching and Computer Vision/OCR components, which are never exposed directly to the user. The **data layer** is a single SQLite database file. Each layer depends only on the layer below it, which keeps modules independently testable and replaceable.

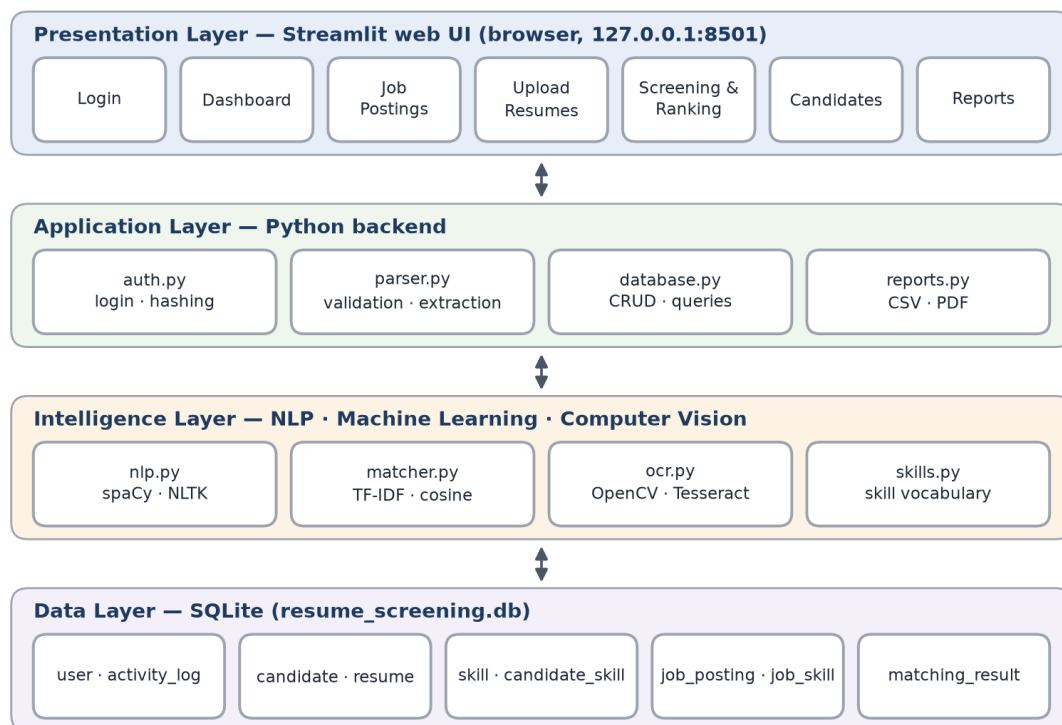


Figure 2: Four-layer system architecture

3.2 Data Flow Diagrams

3.2.1 Level 0 — Context Diagram

The context diagram (Figure 3) treats the whole system as a single process. The recruiter/administrator supplies credentials, job descriptions, resumes and shortlist decisions, and receives ranked candidates, scores, explanations and reports.

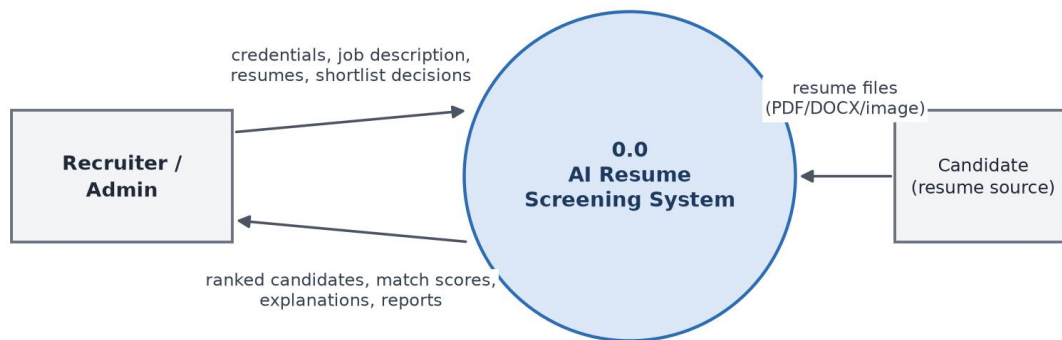


Figure 3: DFD Level 0 (context diagram)

3.2.2 Level 1 DFD

Level 1 (Figure 4) decomposes the system into six processes — authentication, job posting management, upload and text extraction, NLP processing, matching and ranking, and shortlisting and reporting — and four data stores.

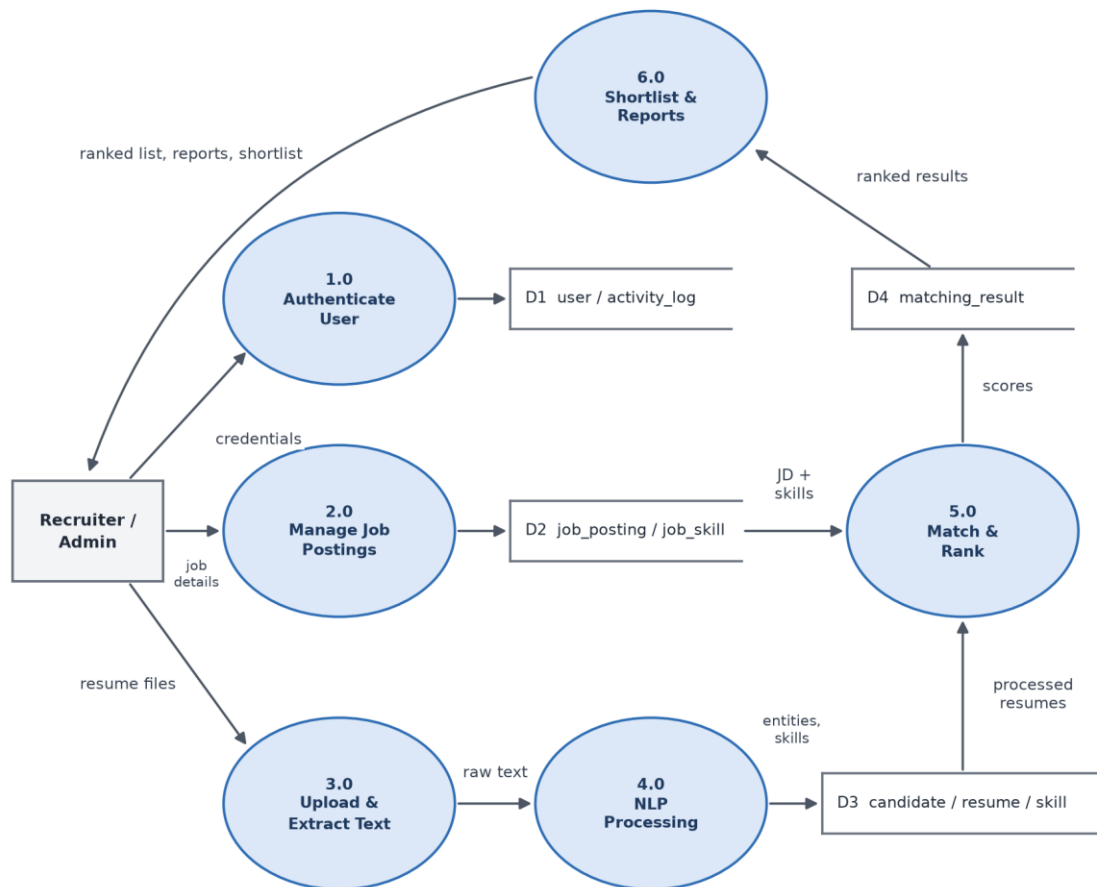


Figure 4: DFD Level 1

3.2.3 Level 2 DFD — NLP, OCR and ML

Level 2 (Figure 5) expands processes 3.0, 4.0 and 5.0. Text from documents with a text layer and text recovered by OCR both flow into the same NLP pipeline, whose output feeds the matching sub-processes.

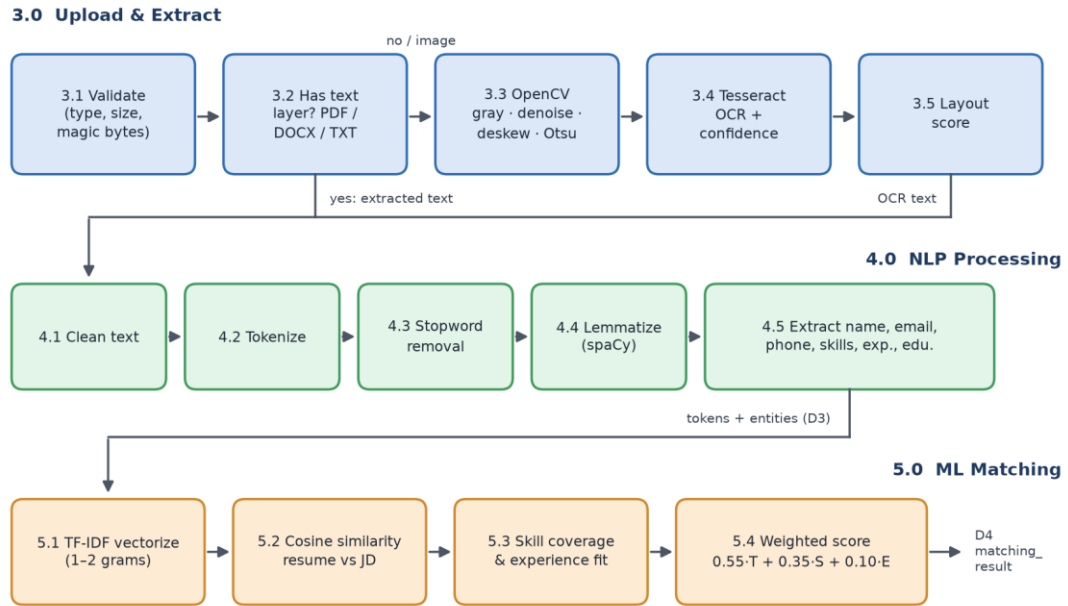


Figure 5: DFD Level 2 — upload/OCR, NLP and ML matching

3.3 Entity Relationship Diagram

The ER diagram (Figure 6) shows nine entities. A USER posts many JOB_POSTINGS and generates many ACTIVITY_LOG entries. A CANDIDATE submits one or more RESUMES and has many SKILLS through the CANDIDATE_SKILL associative entity; a JOB_POSTING requires many SKILLS through JOB_SKILL. MATCHING_RESULT resolves the many-to-many relationship between RESUME and JOB_POSTING and stores the score and selection status.

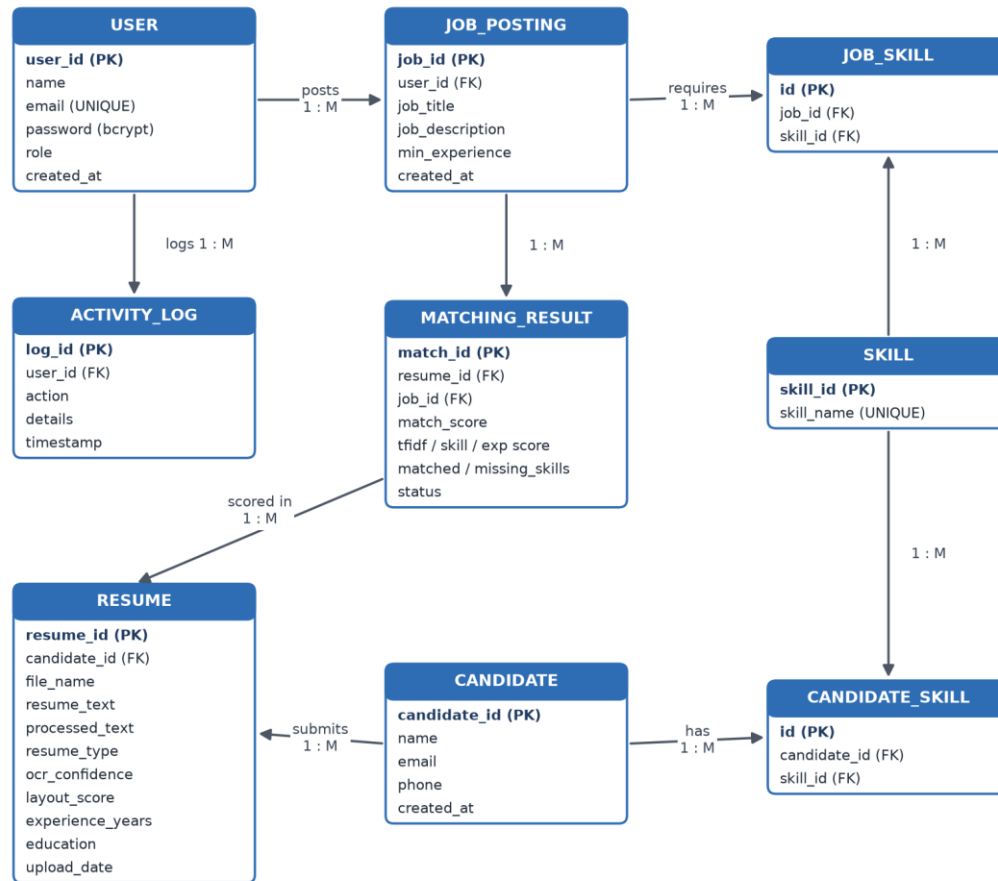


Figure 6: Entity Relationship diagram

3.4 Use Case Diagram

Figure 7 shows the two actors. The **Recruiter** logs in, creates job postings, uploads and processes resumes, views ranked results, shortlists candidates and generates reports. The **Administrator** can perform all recruiter use cases and additionally manage users, view the User Activity Report and review system configuration such as OCR availability.

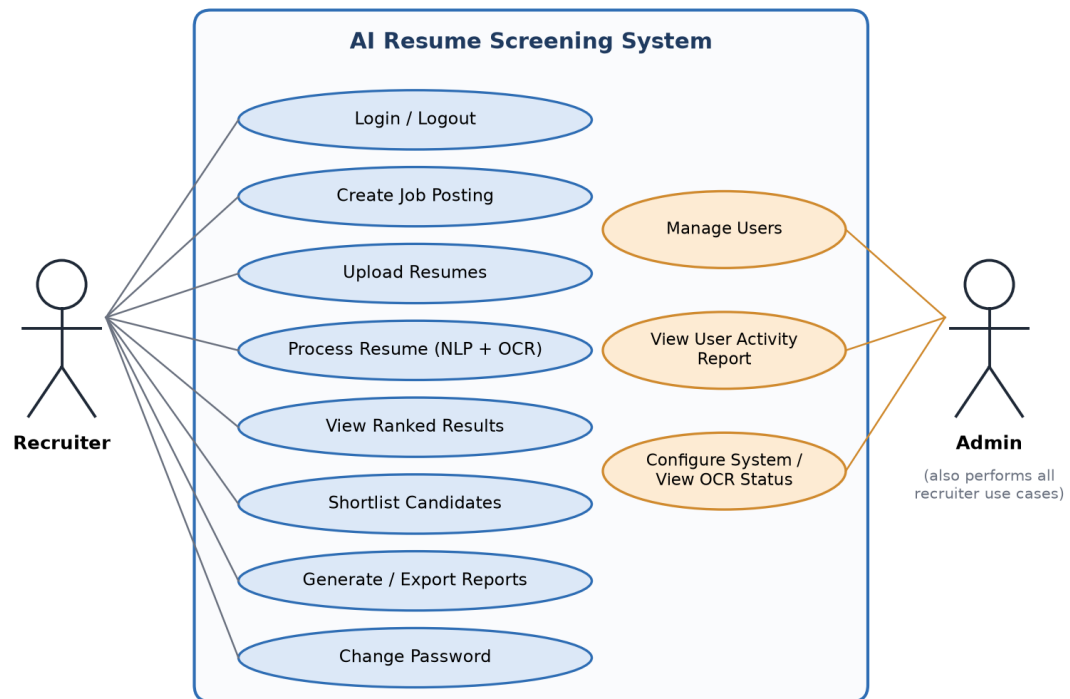


Figure 7: Use case diagram

3.5 Process Flowchart

The end-to-end processing logic for an uploaded resume is shown in Figure 8.

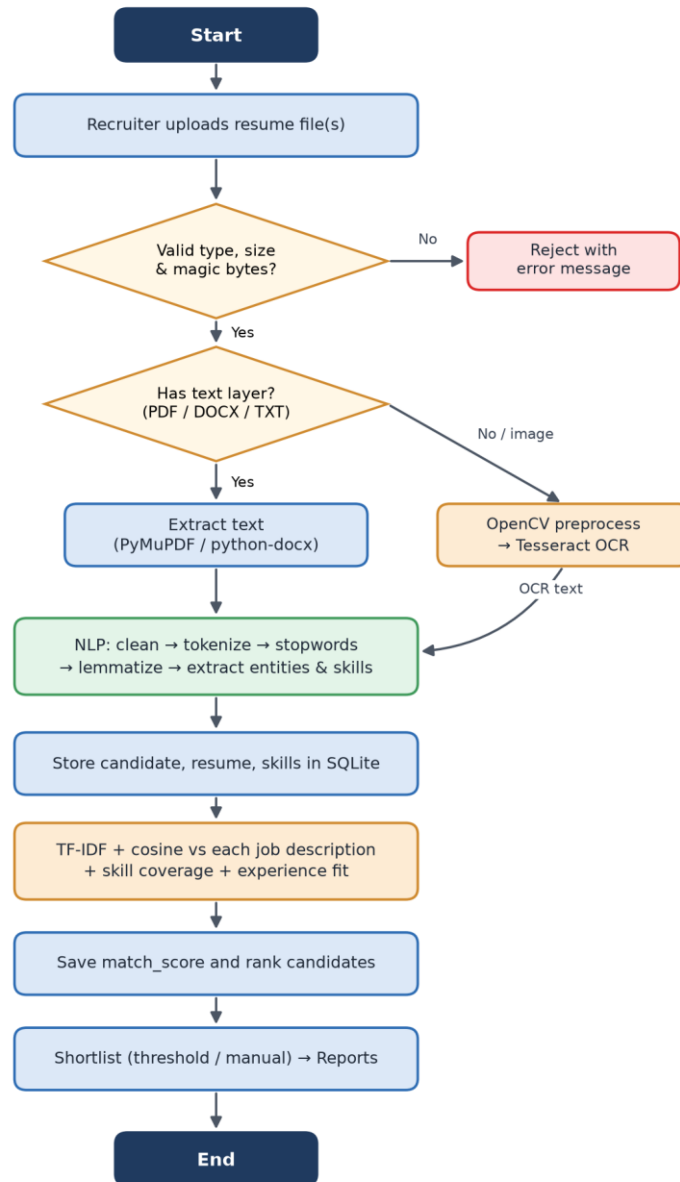


Figure 8: Resume processing flowchart

3.6 Module Design

The system is divided into seven modules, matching the structure proposed in the project synopsis.

Table 5: System modules

#	Module	Responsibility	Source files
1	User Management	Login, logout, bcrypt password hashing, account creation, password change, role checks, activity logging.	core/auth.py, ui/settings_page.py
2	Resume Upload & Parsing	File validation (extension, size, magic bytes); text extraction from PDF, DOCX, TXT; detection of scanned pages.	core/parser.py, ui/upload.py

#	Module	Responsibility	Source files
3	NLP Processing	Cleaning, tokenisation, stopword removal, lemmatisation; extraction of name, e-mail, phone, skills, experience, education, sections.	core/nlp.py, core/skills.py
4	ML Matching	TF-IDF vectorisation, cosine similarity, skill coverage, experience fit, weighted score, ranking and explanation.	core/matcher.py, ui/screening.py
5	Computer Vision & OCR	Grayscale, denoise, deskew, binarise, layout scoring, Tesseract OCR with word confidence.	core/ocr.py
6	Database	Schema creation, transactions, CRUD operations and analytical queries on SQLite.	core/database.py
7	Result & Reporting	Dashboard, ranked results, shortlisting, six reports with CSV/PDF export.	core/reports.py, ui/dashboard.py, ui/reports_page.py

3.7 Database Design

The database is normalised to third normal form. Skills are stored once in the SKILL table and linked to candidates and jobs through associative tables, which eliminates repeating groups and makes skill analytics a simple join. Foreign keys are enforced (PRAGMA foreign_keys = ON) with cascading deletes, and UNIQUE constraints prevent duplicate users, skills and match results. The data dictionary for every table follows.

Table 6: Data dictionary — USER

Field	Data type	Constraint	Description
user_id	INTEGER	PK, auto-increment	Unique identifier of the user
name	TEXT	NOT NULL	Name of recruiter/admin
email	TEXT	NOT NULL, UNIQUE	Login e-mail (stored lower-case)
password	TEXT	NOT NULL	bcrypt hash of the password
role	TEXT	NOT NULL, default recruiter	recruiter or admin
created_at	TEXT	NOT NULL	Account creation timestamp

Table 7: Data dictionary — CANDIDATE

Field	Data type	Constraint	Description
candidate_id	INTEGER	PK, auto-increment	Unique identifier of the candidate
name	TEXT		Candidate name extracted by NLP
email	TEXT		Candidate e-mail
phone	TEXT		Contact number
created_at	TEXT	NOT NULL	Record creation timestamp

Table 8: Data dictionary — SKILL

Field	Data type	Constraint	Description
skill_id	INTEGER	PK, auto-increment	Unique skill identifier
skill_name	TEXT	NOT NULL, UNIQUE	Canonical skill name

Table 9: Data dictionary — CANDIDATE_SKILL

Field	Data type	Constraint	Description
id	INTEGER	PK	Unique identifier
candidate_id	INTEGER	FK → candidate, CASCADE	Links to candidate
skill_id	INTEGER	FK → skill, CASCADE	Links to skill (pair is UNIQUE)

Table 10: Data dictionary — RESUME

Field	Data type	Constraint	Description
resume_id	INTEGER	PK	Unique resume identifier
candidate_id	INTEGER	FK → candidate, CASCADE	Owner of the resume
file_name	TEXT	NOT NULL	Sanitised original file name
resume_text	TEXT		Extracted (cleaned) resume text
processed_text	TEXT		Lemmatised tokens used for TF-IDF
resume_type	TEXT	NOT NULL	text or scanned
upload_date	TEXT	NOT NULL	Upload timestamp
ocr_confidence	REAL		Mean Tesseract word confidence (0–100)
layout_score	REAL		Resume structure / scan layout score (0–100)
experience_years	REAL		Years of experience extracted
education	TEXT		Degrees detected
processing_time	REAL		Parsing time in seconds
uploaded_by	INTEGER	FK → user	User who uploaded the file

Table 11: Data dictionary — JOB_POSTING

Field	Data type	Constraint	Description
job_id	INTEGER	PK	Unique job identifier
user_id	INTEGER	FK → user	Recruiter who posted the job
job_title	TEXT	NOT NULL	Title of the position
job_description	TEXT	NOT NULL	Full job description
min_experience	REAL	default 0	Minimum years of experience

Field	Data type	Constraint	Description
created_at	TEXT	NOT NULL	Creation timestamp

Table 12: Data dictionary — JOB_SKILL

Field	Data type	Constraint	Description
id	INTEGER	PK	Unique identifier
job_id	INTEGER	FK → job_posting, CASCADE	Links to job
skill_id	INTEGER	FK → skill, CASCADE	Links to skill (pair is UNIQUE)

Table 13: Data dictionary — MATCHING_RESULT

Field	Data type	Constraint	Description
match_id	INTEGER	PK	Unique match identifier
resume_id	INTEGER	FK → resume, CASCADE	Resume being scored
job_id	INTEGER	FK → job_posting, CASCADE	Job being matched (resume, job) UNIQUE
match_score	REAL	NOT NULL	Final weighted score (0–100)
tfidf_score	REAL		Content similarity component
skill_score	REAL		Required-skill coverage component
experience_score	REAL		Experience fit component
matched_skills	TEXT		Comma-separated matched skills
missing_skills	TEXT		Comma-separated missing skills
status	TEXT	NOT NULL, default Pending	Pending / Selected / Rejected
processing_time	REAL		Matching time per resume (s)
created_at	TEXT	NOT NULL	Scoring timestamp

Table 14: Data dictionary — ACTIVITY_LOG

Field	Data type	Constraint	Description
log_id	INTEGER	PK	Unique log identifier
user_id	INTEGER	FK → user, SET NULL	Acting user
action	TEXT	NOT NULL	login, upload_resumes, screen_job, set_status, export_report, ...
details	TEXT		Free-text details
timestamp	TEXT	NOT NULL	Time of the action

3.8 User Interface Design

The interface uses a persistent sidebar for navigation and a wide main area. Pages follow the recruiter's workflow: Dashboard, Job Postings, Upload Resumes, Screening & Ranking, Candidates, Reports and Settings. Scores are displayed as progress bars, skills as coloured chips (blue for matched, red for missing), and long outputs such as extracted text are placed in collapsible sections to keep pages readable. Screenshots of every page are presented in Chapter 6.

Chapter 4: Implementation

This chapter describes how the design was implemented. The implementation follows the step-wise methodology of the synopsis: requirement analysis, system design, database design, resume processing and NLP, machine learning, computer vision and OCR, back-end development, front-end development and testing.

4.1 Technology Stack

Table 15: Technology stack and justification

Layer	Technology	Reason for selection
Language	Python 3.13	Richest ecosystem for NLP, ML and CV; rapid development.
User interface	Streamlit 1.59 + Plotly	Pure-Python web UI with widgets, data editors and interactive charts.
NLP	spaCy 3.8, NLTK 3.10	Fast industrial lemmatiser and NER; comprehensive stopword corpus.
Machine learning	scikit-learn 1.9	Robust, well-tested TfidfVectorizer and cosine_similarity.
Computer vision	OpenCV 5	Efficient image denoising, geometric transforms and thresholding.
OCR	Tesseract 5.4 + pytesseract	Accurate open-source LSTM OCR with per-word confidence.
Documents	PyMuPDF, python-docx	Fast PDF text extraction and page rasterisation; DOCX paragraphs and tables.
Database	SQLite 3	Serverless, zero-configuration, ACID-compliant relational database.
Security	bcrypt	Adaptive, salted password hashing resistant to brute force.
Reports	ReportLab, Pandas	Programmatic PDF tables; CSV export and data manipulation.

4.2 Project Structure

The source code is organised into a thin UI package, a core package containing all business and AI logic, scripts, and tests. The project contains 2,335 lines of Python in total.

```

AI-Resume-Screening-System/
├── app.py                # Streamlit entry point
├── config.py             # configuration
├── requirements.txt
├── core/                 # back-end and AI modules
│   ├── auth.py database.py parser.py nlp.py skills.py
│   ├── ocr.py matcher.py reports.py seed.py
│   └── ui/               # Streamlit pages
│       ├── dashboard.py jobs.py upload.py screening.py
│       ├── candidates.py reports_page.py settings_page.py common.py
├── scripts/generate_samples.py
├── sample_data/          # demo resumes and job postings
├── data/                 # resume_screening.db (runtime)
└── tests/test_pipeline.py

```

Table 16: Source files and size

File	Purpose	Lines
app.py	Streamlit entry point, login screen, navigation	76
config.py	Paths, limits, score weights, Tesseract lookup	44
core/auth.py	Authentication and password hashing	55
core/database.py	Schema and all SQL operations	363
core/parser.py	Validation and text extraction	148
core/nlp.py	NLP preprocessing and information extraction	251
core/skills.py	Skill vocabulary and aliases	56
core/ocr.py	OpenCV preprocessing and Tesseract OCR	144
core/matcher.py	TF-IDF matching and scoring	79
core/reports.py	Report queries, CSV and PDF export	125
core/seed.py	Loads the sample dataset	47
ui/*.py (8 files)	Streamlit pages and shared UI helpers	533
scripts/generate_samples.py	Generates the sample resumes and job postings	245
tests/test_pipeline.py	Automated test suite (pytest)	167

4.3 Resume Upload and Parsing Module

Every uploaded file first passes `validate_file()`, which enforces an extension whitelist, rejects empty files and files larger than 10 MB, and compares the first bytes of the file with the expected signature (for example %PDF for PDF and PK for DOCX). A file renamed from `.exe` to `.pdf` is therefore rejected before any parser touches it.

PDF files are opened with PyMuPDF. Each page's text layer is read; if a page yields fewer than 50 characters it is treated as scanned, rendered at 300 DPI and sent to the OCR module. DOCX files are read with `python-docx`, including text inside tables, and TXT files are decoded as UTF-8 with a Latin-1 fallback. The parser records the resume type (text or scanned), OCR confidence and processing time.

4.4 NLP Processing Module

The NLP module performs two jobs: preparing text for vectorisation and extracting structured information.

4.4.1 Text Preprocessing

1. **Cleaning:** bullet glyphs, repeated whitespace and blank lines are removed.
2. **Tokenisation and lemmatisation:** spaCy's `en_core_web_sm` pipeline (parser and NER disabled for speed) tokenises the lower-cased text and supplies the lemma of each token. If spaCy is unavailable the module falls back to NLTK's WordNet lemmatiser.
3. **Filtering:** punctuation, numbers, NLTK English stopwords and domain noise words (resume, curriculum, vitae, e-mail, phone, ...) are removed.
4. **Skill canonicalisation:** canonical names of detected skills are appended as single tokens (e.g. `machine_learning`) so that aliases such as “ML” and “machine learning” share TF-IDF features.

4.4.2 Information Extraction

Table 17: Information extraction techniques

Field	Technique
E-mail	Regular expression for <code>local-part@domain.tld</code> .
Phone	Regular expression for optional country code and 3–5 digit groups; accepted when it contains 10–13 digits.
Name	First short line (2–4 alphabetic words) that is not a section heading; fallback to spaCy PERSON entity; last resort file name.
Skills	Vocabulary of 119 canonical skills with aliases, matched case-insensitively with symbol-aware word boundaries (C++, C#, .NET, Node.js). Ambiguous one-letter languages (C, R, Go) are kept only when listed near a “skills/languages” label.
Experience	Explicit phrases (“5+ years of experience”); otherwise date ranges (e.g. 2019 – Present) inside the Experience section, with overlapping periods merged.
Education	Degree patterns: PhD, M.Tech, MCA, MBA, M.Sc, B.Tech, B.E., BCA, B.Sc, B.Com, Diploma.
Sections	Heading detection for summary, education, experience, skills, projects, certifications, achievements.

Listing 4.1: NLP preprocessing (simplified from core/nlp.py)

```
def preprocess(text: str) -> str:
    sw = stopwords()
    doc = _spacy()(text.lower(), disable=["parser", "ner"])
    tokens = [t.lemma_ for t in doc
               if not t.is_space and not t.is_punct and not t.like_num and t.text not in
sw]
    tokens += [s.replace(" ", "_") for s in extract_skills(text)] # canonical skills
    return " ".join(tokens)
```

4.5 Computer Vision and OCR Module

Scanned resumes pass through the OpenCV pipeline illustrated with a real sample in Figure 9:

1. **Grayscale and upscaling:** colour is discarded; images narrower than 1500 px are upscaled with bicubic interpolation so that glyphs approach 300 DPI.
2. **Denoising:** `cv2.fastNlMeansDenoising` removes speckle noise while preserving character edges.
3. **Deskewing:** a projection-profile method rotates a downsampled binary image through candidate angles (coarse 1° steps within $\pm 10^\circ$, then 0.1° refinement) and chooses the angle that maximises the variance of the horizontal ink profile — level text lines produce the sharpest peaks.
4. **Binarisation:** Otsu's method selects the optimal global threshold.
5. **OCR:** Tesseract (LSTM engine, `--oem 3 --psm 6`) returns words with confidences; lines are reassembled from block, paragraph and line numbers, and the mean word confidence is stored as `ocr_confidence`.
6. **Layout score:** text regions are found by morphological dilation and contour detection. The score (0–100) rewards a sensible ink density, a well-structured number of text blocks, clean margins and low skew. For scanned resumes it is averaged with the content-structure score.

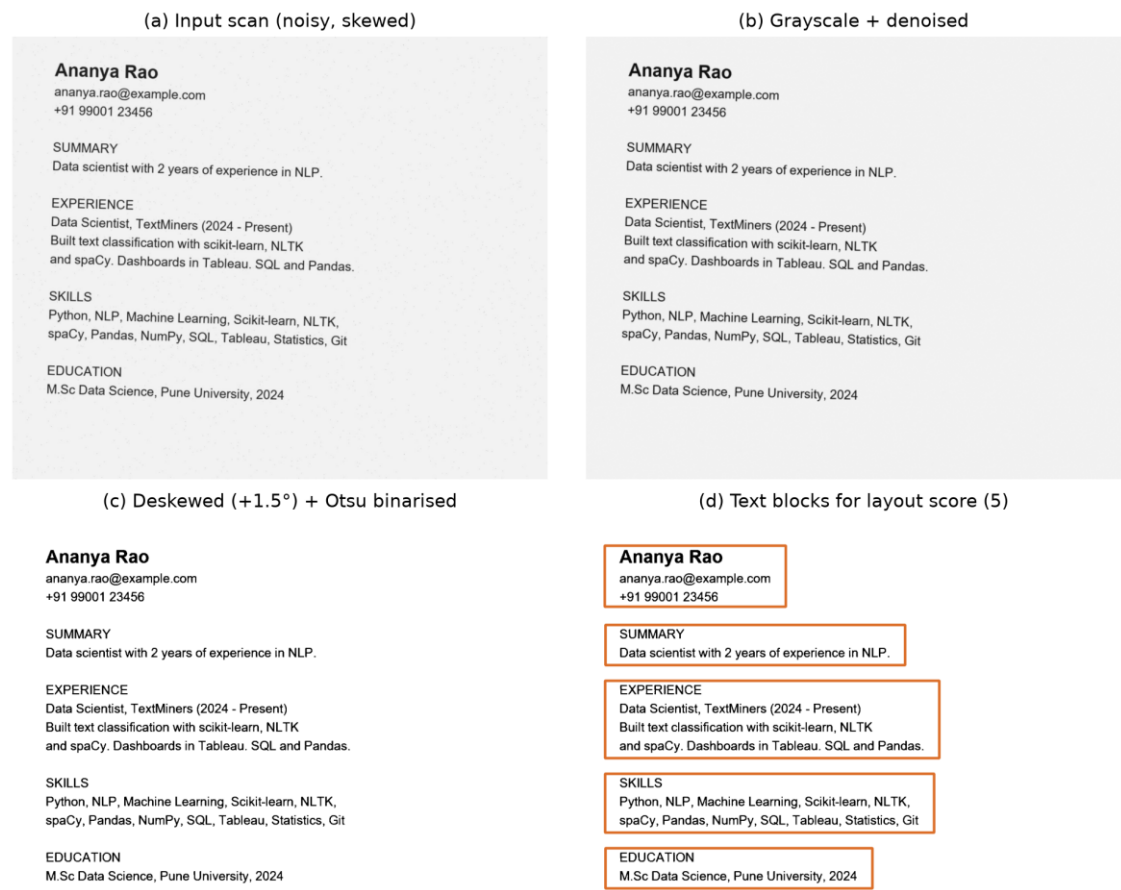


Figure 9: OpenCV preprocessing stages on a real scanned resume

Listing 4.2: Projection-profile skew estimation (simplified from core/ocr.py)

```
def estimate_skew(gray, max_angle=10.0):
    inv = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)[1]
    def score(a): # variance of the horizontal projection after rotating by a degrees
        return np.var(_rotate(inv, a).sum(axis=1))
    best = max(np.arange(-max_angle, max_angle + 0.01, 1.0), key=score)
    best = max(np.arange(best - 1.0, best + 1.01, 0.1), key=score)
    return round(float(best), 2)
```

4.6 Machine Learning Matching Module

For each job, the job description is preprocessed with the same NLP pipeline and a `TfidfVectorizer` (unigrams and bigrams, sub-linear TF) is fitted on the job description together with all resumes. The cosine similarity between the job vector and each resume vector is computed. Because resumes contain much text unrelated to any single job, raw cosine values for strong matches rarely exceed 0.6; the value is therefore divided by 0.6 and capped at 1.

Table 18: Components of the match score

Component	Formula	Weight
-----------	---------	--------

Component	Formula	Weight
Content similarity (T)	$\min(\text{cosine} / 0.6, 1)$	0.55
Skill coverage (S)	$ \text{job skills} \cap \text{resume skills} / \text{job skills} $	0.35
Experience fit (E)	$\min(\text{candidate years} / \text{required years}, 1)$; 1 if no requirement	0.10
Match score	$100 \times (0.55 \cdot T + 0.35 \cdot S + 0.10 \cdot E)$	—

If a job description contains no recognisable skills, the skill weight is added to the content-similarity weight. The weights are defined in `config.py`. Matched and missing skills are stored with every result, and results are upserted so that re-screening updates scores without discarding a recruiter's Selected/Rejected decision.

Listing 4.3: Weighted scoring (simplified from `core/matcher.py`)

```
def score_components(similarity, resume_skills, job_skills, experience, min_experience):
    tfidf = min(similarity / TFIDF_CEILING, 1.0)
    matched = sorted(set(job_skills) & set(resume_skills))
    skills = len(matched) / len(job_skills) if job_skills else None
    exp = min((experience or 0) / min_experience, 1.0) if min_experience else 1.0
    w_t, w_s, w_e = WEIGHT_TFIDF, WEIGHT_SKILLS, WEIGHT_EXPERIENCE
    if skills is None:
        # no skills in JD -> use content only
        w_t, w_s, skills = w_t + w_s, 0.0, 0.0
    return round(100 * (w_t * tfidf + w_s * skills + w_e * exp), 2), matched
```

4.7 Database Module

`core/database.py` creates the schema on start-up and exposes functions for every operation. A context manager opens a connection, enables foreign keys, commits on success and rolls back on any exception, so a candidate, their resume and their skill links are always inserted atomically. All statements use `?` placeholders, and analytical views (ranked results, skill aggregation, report data) are returned as Pandas DataFrames.

4.8 User Management and Authentication

Passwords are hashed with `bcrypt.hashpw` using a random salt and verified with `bcrypt.checkpw`; plain-text passwords are never stored or logged. On the first run a default administrator is created and the login screen prompts for the password to be changed. Registration enforces a valid e-mail, a unique account and a password of at least eight characters containing letters and digits. Successful and failed logins, uploads, screenings, status changes and exports are written to the activity log.

4.9 Result and Reporting Module

The dashboard shows key metrics and charts. The Screening & Ranking page shows a bar chart with the shortlist threshold, an editable ranked table and an explanation panel per candidate. The Reports page produces the six reports listed in the synopsis — User Activity (administrators only), Candidate, Resume Analysis, Matching Result, Job Posting and System Performance — each downloadable as CSV or as a formatted landscape PDF generated with ReportLab.

Chapter 5: Testing

Testing verified that each module works correctly in isolation, that the modules work together, and that the complete application meets the requirements from the user's point of view.

5.1 Testing Strategy

Table 19: Levels of testing performed

Level	Approach	Tools
Unit testing	Individual functions (extractors, validators, deskew, scoring) tested with known inputs and expected outputs.	pytest
Integration testing	Parser → NLP → database → matcher → reports tested on the full sample dataset in a temporary database.	pytest
System testing	The running Streamlit application exercised page by page, including uploads and OCR.	Streamlit AppTest, Playwright browser automation
User acceptance testing	Workflow test cases executed from the recruiter and administrator perspective (Section 5.3).	Streamlit AppTest
Regression testing	Full automated suite re-run after every defect fix.	pytest

5.2 Automated Test Suite

The automated suite in `tests/test_pipeline.py` runs against a temporary database so production data is never modified. All 12 tests passed in the final run.

Table 20: Automated test results

ID	Test	What is verified	Result
AT-01	NLP extraction	Name, e-mail, phone, skills incl. aliases (ML, C++), experience from date range, education, stopword removal	Pass
AT-02	Experience phrase & ambiguous skills	"5+ years of professional experience" → 5; "Plan C", "Go ahead" not detected as skills	Pass
AT-03	File validation	.exe rejected; fake PDF (wrong signature) rejected; valid TXT accepted	Pass
AT-04	Multi-format parsing	Sample PDF, DOCX and TXT resumes parsed with e-mail and skills	Pass
AT-05	OpenCV preprocessing	Binary output, small residual skew, layout score within 0–100	Pass
AT-06	Deskew (−5°)	Correction $\approx +5^\circ$, residual skew $\leq 0.3^\circ$	Pass
AT-07	Deskew (−1.5°)	Correction $\approx +1.5^\circ$, residual skew $\leq 0.3^\circ$	Pass
AT-08	Deskew (+1.5°)	Correction $\approx -1.5^\circ$, residual skew $\leq 0.3^\circ$	Pass
AT-09	Deskew (+5°)	Correction $\approx -5^\circ$, residual skew $\leq 0.3^\circ$	Pass

ID	Test	What is verified	Result
AT-10	OCR end-to-end	Scanned PNG resume recognised as scanned; OpenCV skill extracted	Pass
AT-11	Matching quality	Relevant resume scores > 60 and ranks above an unrelated resume; no missing skills	Pass
AT-12	Auth + database + reports	bcrypt hashing, login success/failure, duplicate account, sample load, correct top candidate per job, auto-shortlist, all 6 reports to PDF and CSV	Pass

5.3 Test Cases

The following functional test cases were executed against the running application. The actual results are those observed during execution.

Table 21: Functional test cases

ID	Test case / input	Expected result	Actual result	Status
TC-01	Login with admin@resumeai.local and correct password	Dashboard opens	Dashboard displayed with metrics	Pass
TC-02	Login with correct e-mail and wrong password	Access denied with message	"Invalid email or password." shown; attempt logged	Pass
TC-03	Admin creates user with password "short"	Rejected by validation	"Password must be at least 8 characters and contain letters and digits."	Pass
TC-04	Admin creates recruiter rec1@example.com / Recruit123	Account created, password hashed	"User rec1@example.com created."	Pass
TC-05	Change password with wrong current password	Rejected	"Current password is incorrect."	Pass
TC-06	Type JD: Python developer with Django, REST API, PostgreSQL, Docker, Git; 2+ years	Skills and experience auto-detected	django, docker, git, postgresql, python, rest api; min. experience 2.0	Pass
TC-07	Save the job with "screen existing resumes" ticked	Job stored and all resumes scored	"Job saved. 7 resume(s) screened."; top candidate Rahul Mehta (60.6 %)	Pass
TC-08	Recruiter opens User Activity Report	Access restricted to admin	"available to administrators only" warning	Pass
TC-09	Recruiter opens Settings	User management hidden	Only password change and system info shown	Pass
TC-10	Save job with empty title/description	Validation error	"Please provide a title and a job description of at least 30 characters."	Pass
TC-11	Upload scanned JPG resume (ananya_rao_scan.jpg)	OCR extracts details	Type Scanned; name, e-mail, phone, M.Sc, 2 yrs, 13 skills; OCR confidence 94 %	Pass
TC-12	Upload a file named .pdf that is not a PDF	Rejected before parsing	"file content does not match its .pdf extension"	Pass
TC-13	Upload a .exe file	Rejected	Rejected: "unsupported file type .exe"	Pass
TC-14	Auto-shortlist Full Stack Developer at 60 %	≥ 60 % Selected, others Rejected	Rahul Mehta Selected; remaining candidates Rejected	Pass
TC-15	Export each of the six reports as CSV and PDF	Valid files generated	All reports exported; PDFs start with %PDF signature	Pass

5.4 Extraction Accuracy on the Evaluation Dataset

The evaluation dataset contains seven resumes in five formats (three PDF, one DOCX, one TXT, one PNG scan and one JPEG scan with noise, blur and a 1.5° rotation). For each resume five fields — name, e-mail, phone, years of experience and education — were compared with the known ground truth. **35 of 35 fields (100 %) were extracted correctly.** The first text resume includes the one-time loading of the spaCy language model (≈ 4 s); subsequent text resumes are parsed in well under half a second.

Table 22: Extraction results per resume

Candidate	Format	Fields correct	Skills found	OCR conf.	Parse time
Ananya Rao	JPG (scanned)	5 / 5	13	94.2 %	4.72 s
Vikram Singh	PNG (scanned)	5 / 5	12	94.2 %	4.78 s
Sneha Iyer	TXT (text)	5 / 5	9	—	0.03 s
Rahul Mehta	PDF (text)	5 / 5	21	—	0.04 s
Priya Verma	DOCX (text)	5 / 5	12	—	0.27 s
Neha Kapoor	PDF (text)	5 / 5	7	—	0.03 s
Aarav Sharma	PDF (text)	5 / 5	25	—	4.00 s

Note: the dataset is small and synthetic, so these figures demonstrate correctness of the pipeline rather than a statistically general accuracy. Real-world resumes with multi-column layouts, tables or photographs will be harder; evaluation on a larger labelled dataset is listed as future work.

5.5 Defects Found and Fixed

Testing uncovered several defects, all of which were corrected and covered by regression tests.

Table 23: Defect log

#	Defect	Root cause	Fix
D-01	Phone numbers written as “+91 98765 43210” were not extracted.	Pattern required three digit groups after the country code.	Made the final group optional and validated by digit count (10–13).
D-02	“B.E.” degree not detected.	Word boundary after the trailing dot never matched.	Added a dedicated B.E. pattern.

#	Defect	Root cause	Fix
D-03	Deskew doubled the skew for pages rotated one way and ignored the other direction.	minAreaRect received (row, col) instead of (x, y) coordinates, flipping the angle sign.	Replaced with projection-profile skew estimation; added four parametrised tests. OCR then read “ananya.rao@” correctly (previously “ananya.rac@”) and confidence rose from 93.5 % to 94.2 %.
D-04	PDF report generation failed with missing values.	pandas 3 keeps missing values as floats after astype(str).	Convert each cell explicitly, treating NaN/None as empty.
D-05	Resume type “Scanned” truncated to “Scan...” on the upload page.	Metric column too narrow.	Re-balanced column widths.

Chapter 6: Results and Discussion

6.1 Application Screens

The screenshots below were captured from the running application with the evaluation dataset loaded.

6.1.1 Login

The login page (Figure 10) is the only page available without authentication.

Deploy :

**AI Resume Screening**
NLP · Machine Learning · Computer Vision (OCR)

Email

admin@resumeai.local

Password

Press Enter to submit form

Sign in

First run? Sign in with [admin@resumeai.local](#) / [Admin@123](#) and change the password under Settings.

Figure 10: Login screen

6.1.2 Dashboard

The dashboard (Figures 11 and 12) summarises candidates, scanned resumes, job postings, matches and selections, and visualises the most common skills, the distribution of match scores, the share of resume formats and the screening status by job.

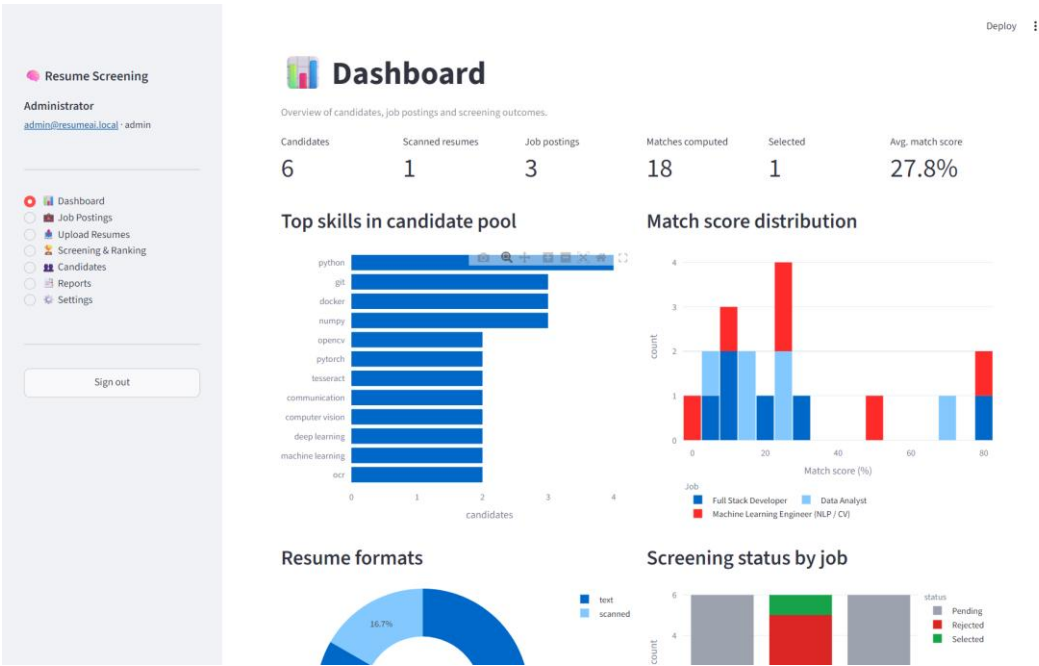


Figure 11: Dashboard — key metrics, top skills and score distribution

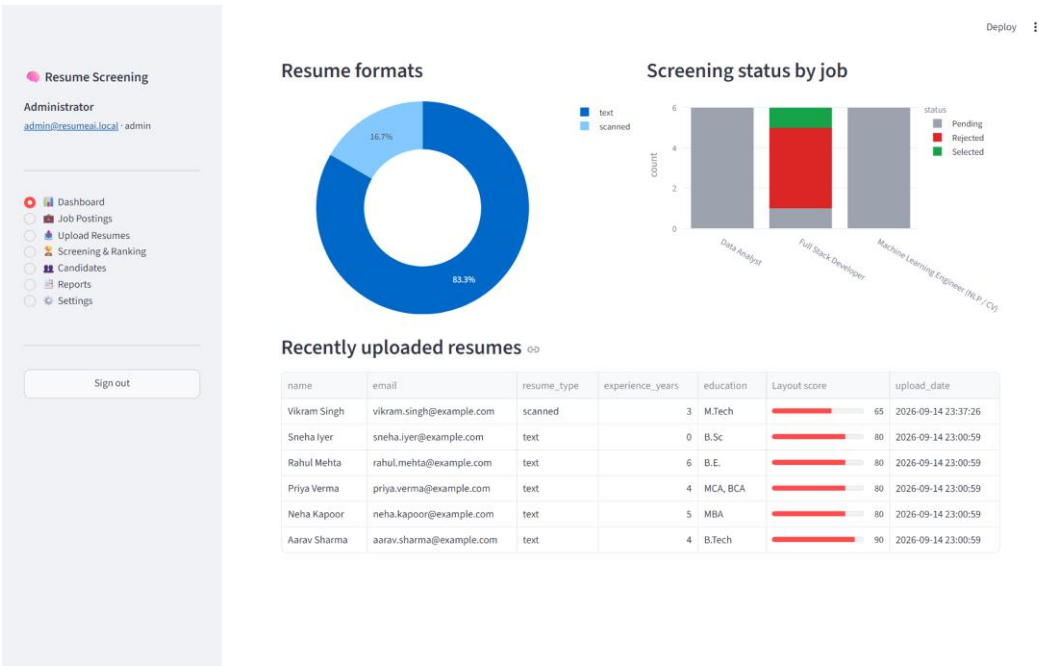


Figure 12: Dashboard — resume formats, status by job and recent uploads

6.1.3 Job Postings

Recruiters create job postings and see the automatically detected skills as chips (Figure 13). Each posting can be re-screened or deleted.

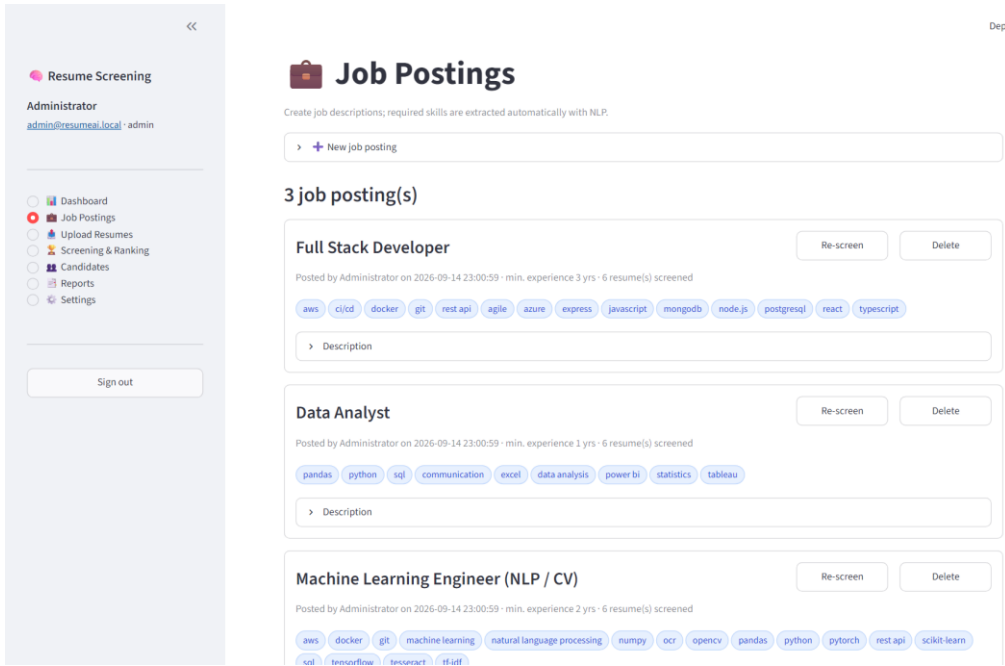


Figure 13: Job Postings page

6.1.4 Upload and OCR

Figure 14 shows a noisy, rotated scanned JPEG processed through OpenCV and Tesseract. The system identified the file as scanned, extracted the candidate’s contact details, education, experience and 13 skills, and reported an OCR confidence of 94 %. The raw text and the preprocessed tokens can be inspected.

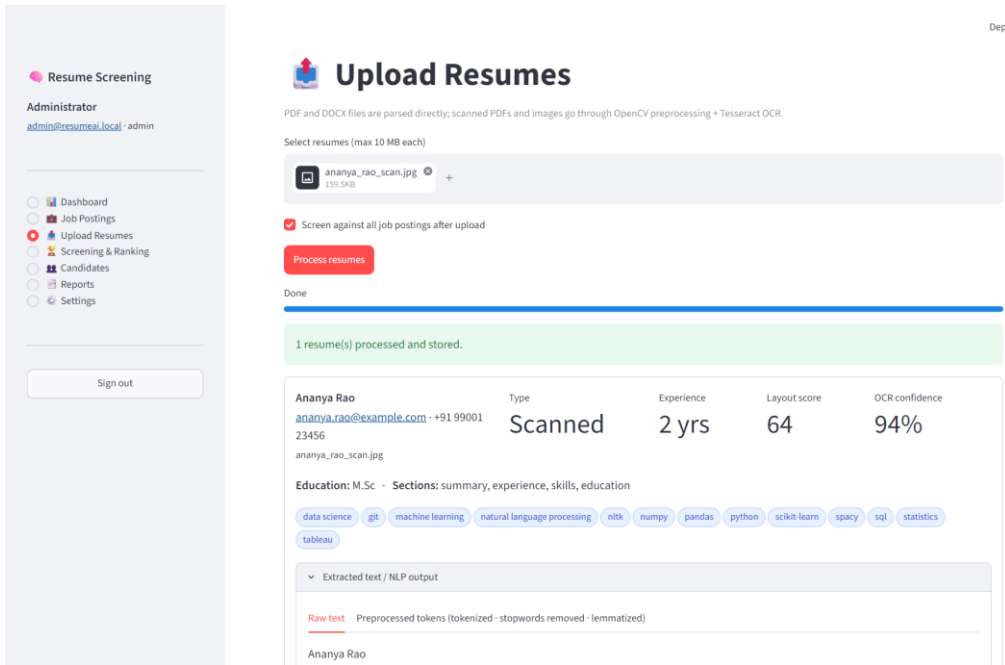
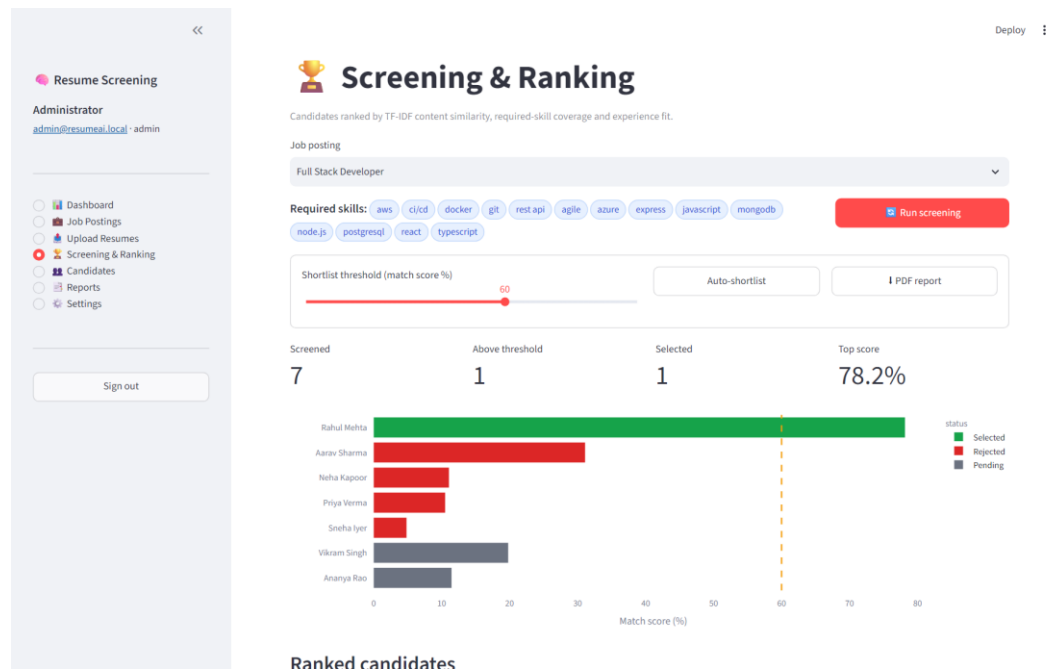


Figure 14: Upload Resumes — scanned resume processed with OCR

6.1.5 Screening and Ranking

The screening page (Figures 15–17) shows the required skills, a shortlist threshold slider, a ranked bar chart, an editable table in which the recruiter can change a candidate’s status, and an explanation panel with component scores and matched/missing skills.



Ranked candidates

Figure 15: Screening & Ranking — threshold and ranked chart

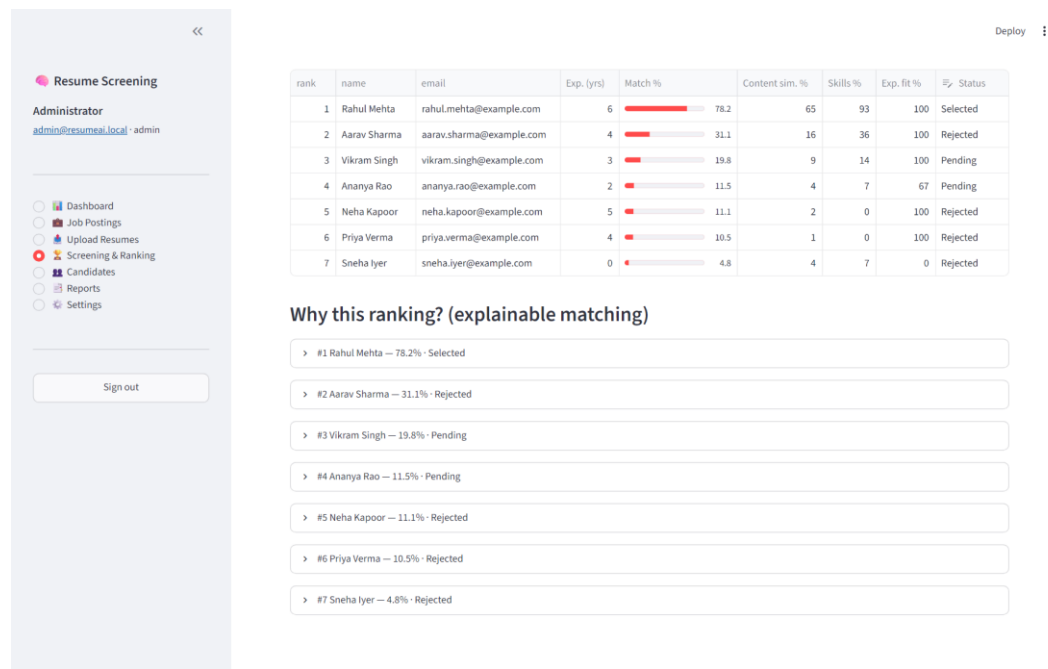


Figure 16: Ranked candidates table with editable status

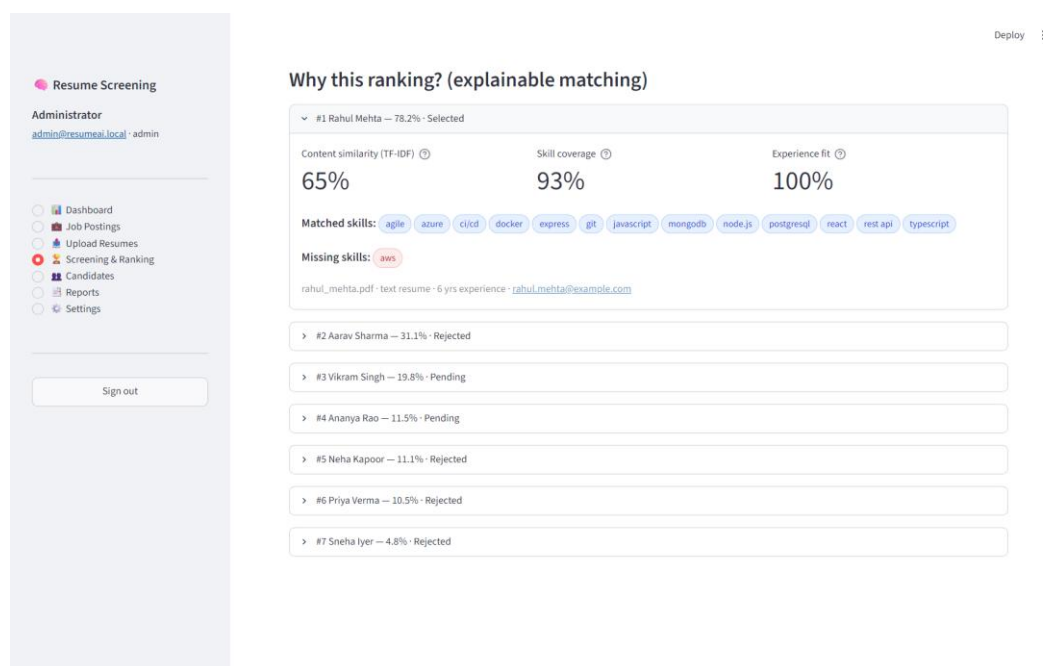


Figure 17: Explainable matching — component scores, matched and missing skills

6.1.6 Candidates

The Candidates page (Figure 18) lets recruiters search the pool, inspect a profile, and see recommended job roles for the candidate — an implementation of the resume-recommendation idea from the synopsis.

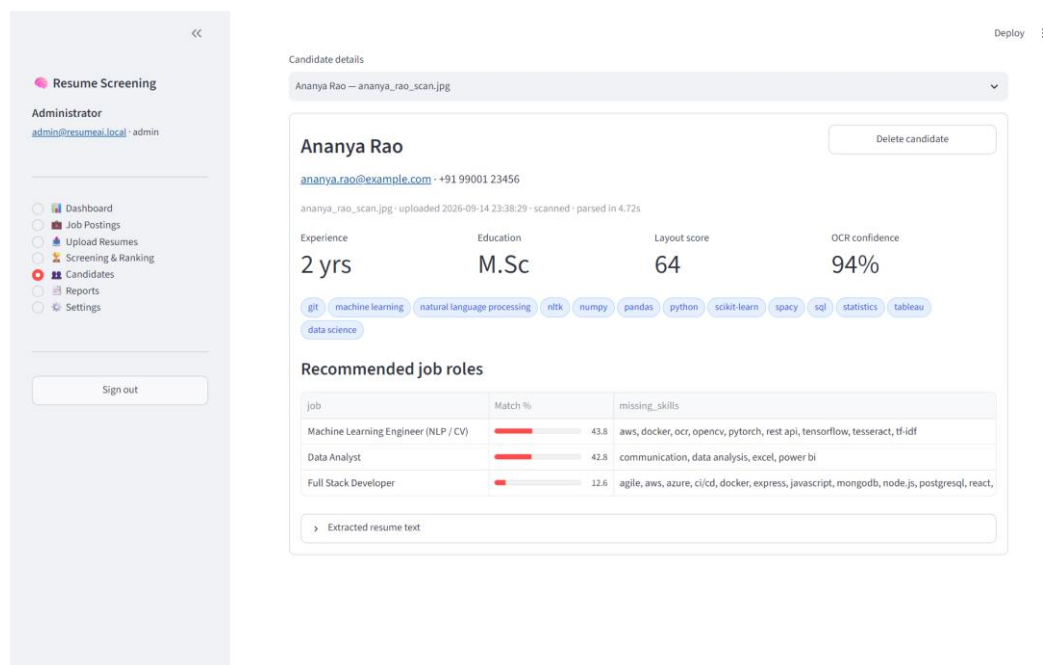
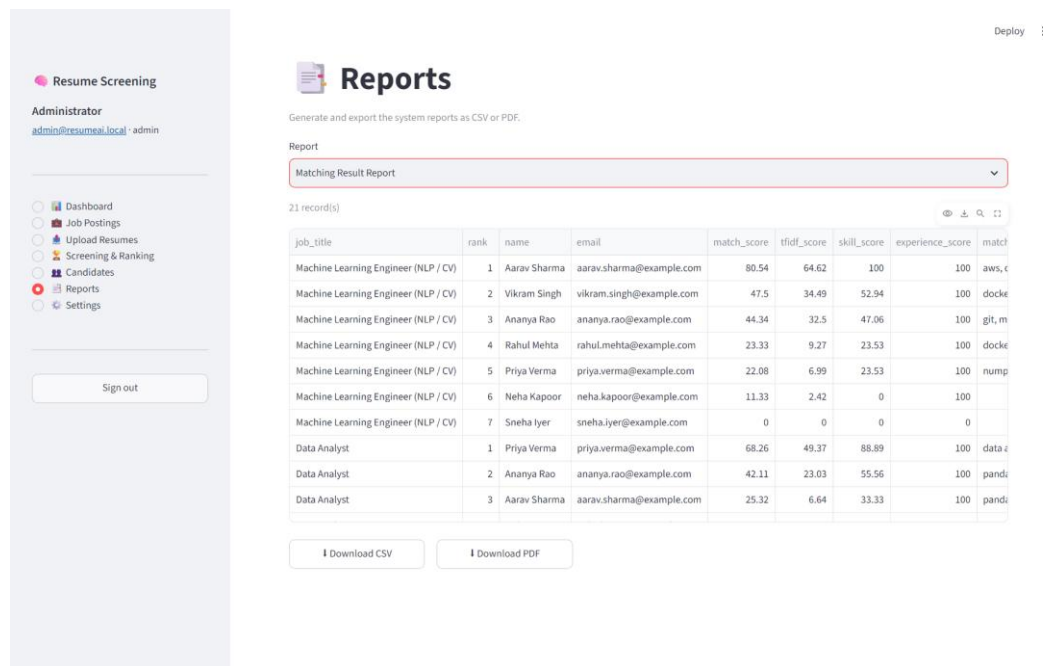


Figure 18: Candidate profile with recommended job roles

6.1.7 Reports and Settings



Resume Screening
Administrator
admin@resumear.local · admin

- Dashboard
- Job Postings
- Upload Resumes
- Screening & Ranking
- Candidates
- Reports**
- Settings

Sign out

Reports

Generate and export the system reports as CSV or PDF.

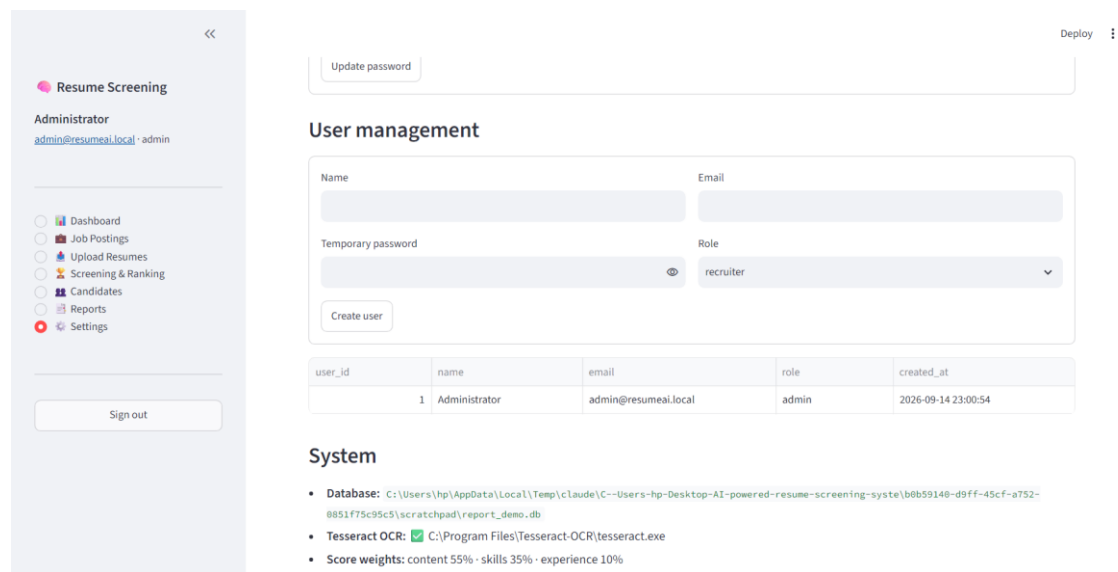
Report
Matching Result Report

21 record(s)

job_title	rank	name	email	match_score	tfidf_score	skill_score	experience_score	match
Machine Learning Engineer (NLP / CV)	1	Aarav Sharma	aarav.sharma@example.com	80.54	64.62	100	100	aws, c
Machine Learning Engineer (NLP / CV)	2	Vikram Singh	vikram.singh@example.com	47.5	34.49	52.94	100	docke
Machine Learning Engineer (NLP / CV)	3	Ananya Rao	ananya.rao@example.com	44.34	32.5	47.06	100	git, m
Machine Learning Engineer (NLP / CV)	4	Rahul Mehta	rahul.mehta@example.com	23.33	9.27	23.53	100	docke
Machine Learning Engineer (NLP / CV)	5	Priya Verma	priya.verma@example.com	22.08	6.99	23.53	100	nump
Machine Learning Engineer (NLP / CV)	6	Neha Kapoor	neha.kapoor@example.com	11.33	2.42	0	100	
Machine Learning Engineer (NLP / CV)	7	Sneha Iyer	sneha.iyer@example.com	0	0	0	0	
Data Analyst	1	Priya Verma	priya.verma@example.com	68.26	49.37	88.89	100	data i
Data Analyst	2	Ananya Rao	ananya.rao@example.com	42.11	23.03	55.56	100	pandi
Data Analyst	3	Aarav Sharma	aarav.sharma@example.com	25.32	6.64	33.33	100	pandi

Download CSV Download PDF

Figure 19: Reports page — Matching Result Report with CSV/PDF export



Resume Screening
Administrator
admin@resumear.local · admin

- Dashboard
- Job Postings
- Upload Resumes
- Screening & Ranking
- Candidates
- Reports
- Settings**

Sign out

Update password

User management

Name Email

Temporary password Role

Create user

user_id	name	email	role	created_at
1	Administrator	admin@resumear.local	admin	2026-09-14 23:00:54

System

- Database:** C:\Users\hp\AppData\Local\Temp\claude\C--Users-hp-Desktop-AI-powered-resume-screening-syste\b8b59148-d9ff-45cf-a752-8851f75c35c5\scratchpad\report_demo.db
- Tesseract OCR:** ☒ C:\Program Files\Tesseract-OCR\tesseract.exe
- Score weights:** content 55% · skills 35% · experience 10%

Figure 20: Settings — password change and user management (admin)

6.2 Ranking Results

Figure 21 shows the match score of every candidate against every job posting, and the tables below list the complete rankings.

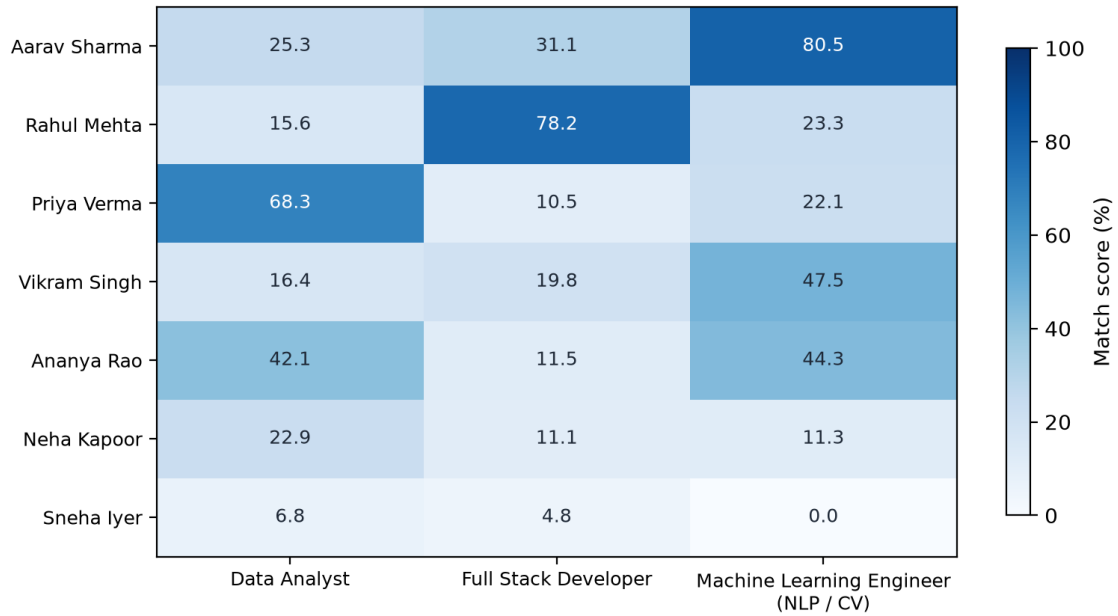


Figure 21: Match scores of all candidates against all job postings

Table 24: Ranking for “Machine Learning Engineer (NLP / CV)” (min. 2 yrs)

Rank	Candidate	Score %	Content %	Skills %	Exp. %	Missing skills
1	Aarav Sharma	80.5	65	100	100	—
2	Vikram Singh	47.5	34	53	100	aws, natural language processing, pandas, rest api, scikit-learn, sql, tensorflow, tf-idf
3	Ananya Rao	44.3	33	47	100	aws, docker, ocr, opencv, pytorch, rest api, tensorflow, tesseract, tf-idf
4	Rahul Mehta	23.3	9	24	100	aws, machine learning, natural language processing, numpy, ocr, opencv, pandas, pytorch, scikit-learn, sql, tensorflow, tesseract, tf-idf
5	Priya Verma	22.1	7	24	100	aws, docker, git, machine learning, natural language processing, ocr, opencv, pytorch, rest api, scikit-learn, tensorflow, tesseract, tf-idf
6	Neha Kapoor	11.3	2	0	100	aws, docker, git, machine learning, natural language processing, numpy, ocr, opencv, pandas, python, pytorch, rest api, scikit-learn, sql, tensorflow, tesseract, tf-idf
7	Sneha Iyer	0.0	0	0	0	aws, docker, git, machine learning, natural language processing, numpy, ocr, opencv, pandas, python, pytorch, rest api, scikit-learn, sql, tensorflow, tesseract, tf-idf

Table 25: Ranking for “Data Analyst” (min. 1 yrs)

Rank	Candidate	Score %	Content %	Skills %	Exp. %	Missing skills
1	Priya Verma	68.3	49	89	100	communication

Rank	Candidate	Score %	Content %	Skills %	Exp. %	Missing skills
2	Ananya Rao	42.1	23	56	100	communication, data analysis, excel, power bi
3	Aarav Sharma	25.3	7	33	100	communication, data analysis, excel, power bi, statistics, tableau
4	Neha Kapoor	22.9	9	22	100	data analysis, pandas, power bi, python, sql, statistics, tableau
5	Vikram Singh	16.4	5	11	100	communication, data analysis, excel, pandas, power bi, sql, statistics, tableau
6	Rahul Mehta	15.6	3	11	100	communication, data analysis, excel, pandas, power bi, sql, statistics, tableau
7	Sneha Iyer	6.8	5	11	0	data analysis, excel, pandas, power bi, python, sql, statistics, tableau

Table 26: Ranking for “Full Stack Developer” (min. 3 yrs)

Rank	Candidate	Score %	Content %	Skills %	Exp. %	Missing skills
1	Rahul Mehta	78.2	65	93	100	aws
2	Aarav Sharma	31.1	16	36	100	agile, azure, express, javascript, mongodb, node.js, postgresql, react, typescript
3	Vikram Singh	19.8	9	14	100	agile, aws, azure, ci/cd, express, javascript, mongodb, node.js, postgresql, react, rest api, typescript
4	Ananya Rao	11.5	4	7	67	agile, aws, azure, ci/cd, docker, express, javascript, mongodb, node.js, postgresql, react, rest api, typescript
5	Neha Kapoor	11.1	2	0	100	agile, aws, azure, ci/cd, docker, express, git, javascript, mongodb, node.js, postgresql, react, rest api, typescript
6	Priya Verma	10.5	1	0	100	agile, aws, azure, ci/cd, docker, express, git, javascript, mongodb, node.js, postgresql, react, rest api, typescript
7	Sneha Iyer	4.8	4	7	0	agile, aws, azure, ci/cd, docker, express, git, mongodb, node.js, postgresql, react, rest api, typescript

6.3 System Performance

Table 27: System Performance Report (generated by the application)

Metric	Value
Total resumes processed	7
Text-based resumes	5
Scanned resumes (OCR)	2
Avg. parsing time - text (s)	0.872

Metric	Value
Avg. parsing time - scanned (s)	4.752
Avg. OCR confidence (%)	94.2
Avg. layout score	77.0
Avg. skills per candidate	14.1
Job postings	3
Matches computed	21
Avg. matching time per resume (s)	0.0022
Avg. match score	28.26
Candidates selected	1
Selection rate (%)	4.8

6.4 Discussion

- **Correct top candidate for every job.** The ML engineer (Aarav Sharma, 80.5 %), the data analyst (Priya Verma, 68.3 %) and the full-stack developer (Rahul Mehta, 78.2 %) each ranked first for the matching role by a wide margin over the second-placed candidate.
- **Sensible partial matches.** The computer-vision engineer (Vikram Singh) and the NLP data scientist (Ananya Rao) — both scanned resumes — ranked second and third for the ML role, reflecting real but incomplete skill overlap. Ananya Rao also ranked second for Data Analyst because of her SQL, Pandas, Tableau and statistics skills.
- **Experience fit works as intended.** The fresh graduate (Sneha Iyer) receives 0 % experience fit for all roles, which lowers her score only by the 10 % experience weight.
- **OCR does not disadvantage candidates.** Scanned resumes produced the same quality of extracted fields as text resumes, at the cost of about 5 seconds of processing per page.
- **Explainability.** Missing-skill lists (e.g. “aws” for the full-stack developer) give recruiters an immediate, auditable reason for each score.
- **Score scale.** Absolute scores are moderate because resumes contain content unrelated to any single job; the threshold slider lets recruiters calibrate the shortlist cut-off for each role.

Chapter 7: Security Implementation

The system handles personal data of candidates and credentials of recruiters, so security mechanisms were implemented at every level of the architecture.

Table 28: Security mechanisms at various levels

Level	Threat	Mechanism implemented
User authentication	Unauthorised access; password theft	Login required for every page; bcrypt salted hashes (never plain text); password strength rules; Streamlit session state for authenticated sessions; logout clears the session.
Authorisation	Privilege misuse	Role-based access: only administrators can create users or view the User Activity Report.
Application	Malicious or malformed uploads	Extension whitelist, 10 MB size limit, magic-byte signature check, safe file-name sanitisation; files parsed in memory and never executed; per-file error handling prevents crashes and information leaks.
Database	SQL injection; inconsistent data	Parameterised queries throughout; foreign keys with cascades; UNIQUE constraints; transactions with rollback; database file stored locally.
Network	Eavesdropping; remote access	Application bound to localhost (127.0.0.1) during development; HTTPS through a reverse proxy recommended for deployment.
ML / NLP	Tampering with processing logic	Models and logic reside only in the back end; users interact through the UI and cannot access internal modules; input text is cleaned before processing.
CV / OCR	Malicious image files	Images decoded with OpenCV after signature validation; OCR runs locally; no external execution of uploaded content.
Audit	Repudiation	Activity log of logins (including failures), uploads, screenings, status changes, user creation and report exports.

Because all processing is local and no third-party AI service is called, candidate data never leaves the organisation's machine — an important privacy property for recruitment data.

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

This project successfully designed and implemented an AI Powered Resume Screening System that integrates Natural Language Processing, Machine Learning and Computer Vision in a single, easy-to-use application. The system:

- extracts text from PDF, DOCX, TXT and scanned image resumes, using OpenCV preprocessing and Tesseract OCR where necessary;
- applies tokenisation, stopwords removal and lemmatisation, and extracts contact details, skills, experience and education;
- matches resumes to job descriptions with TF-IDF and cosine similarity combined with skill coverage and experience fit;
- ranks candidates with transparent, explainable scores and supports automatic and manual shortlisting;
- stores all data in a normalised SQLite database and produces six reports in CSV and PDF format;
- protects data through hashed passwords, role-based access, validated uploads and parameterised queries.

Evaluation on the sample dataset showed correct extraction of all checked fields, OCR confidence above 94 % on scanned resumes, and the intended best-fit candidate ranked first for every job. The project demonstrates the practical value of AI in recruitment: it reduces manual effort, improves consistency and ensures that candidates are not overlooked because of the format of their resume.

8.2 Limitations

- TF-IDF measures lexical rather than semantic similarity; related concepts with no shared vocabulary are not matched unless they are in the skill vocabulary.
- The skill vocabulary is curated manually and must be extended for new domains.
- Complex multi-column layouts, tables and photographs in resumes can reduce extraction and OCR quality.
- Evaluation was performed on a small synthetic dataset.

- SQLite and a single Streamlit process are suited to a single team, not to large concurrent workloads.
- Only English resumes are supported.

8.3 Future Scope

1. **Cloud deployment** on AWS, Azure or Google Cloud with PostgreSQL and HTTPS for organisation-wide access.
2. **Transformer-based semantic matching** using BERT or sentence-embedding models to capture meaning beyond shared keywords.
3. **Named Entity Recognition models** trained on resumes for more robust extraction of organisations, job titles, dates and skills.
4. **Integration with job portals and e-mail** to import resumes and job postings automatically.
5. **Multilingual resume processing** with language detection and multilingual OCR models.
6. **Recruitment analytics** dashboards showing hiring funnels, skill gaps and time-to-hire trends.
7. **Bias auditing and fairness controls**, such as hiding personal attributes during screening.
8. **Feedback learning**, using recruiters' Selected/Rejected decisions to tune score weights for each role.

References

1. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
2. Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed. draft). Stanford University.
3. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
4. Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62–66.
5. Smith, R. (2007). An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 629–633.
6. Buades, A., Coll, B., & Morel, J.-M. (2005). A non-local algorithm for image denoising. *IEEE CVPR*, 60–65.
7. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength Natural Language Processing in Python*.
9. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
10. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
11. Provos, N., & Mazières, D. (1999). A future-adaptable password scheme (bcrypt). *USENIX Annual Technical Conference*.
12. Streamlit documentation — <https://docs.streamlit.io>
13. SQLite documentation — <https://www.sqlite.org/docs.html>
14. Tesseract OCR documentation — <https://tesseract-ocr.github.io>
15. OWASP Foundation. *File Upload Cheat Sheet* and *Password Storage Cheat Sheet* — <https://cheatsheetseries.owasp.org>

Appendix A: Installation and User Manual

A.1 Installation

1. Install Python 3.10 or later.
2. Install Tesseract OCR (Windows: `winget install UB-Mannheim.TesseractOCR`; Ubuntu: `sudo apt install tesseract-ocr`; macOS: `brew install tesseract`).
3. Open a terminal in the project folder and install dependencies: `pip install -r requirements.txt`.
4. Download language resources: `python -m spacy download en_core_web_sm` and `python -c "import nltk; nltk.download('stopwords'); nltk.download('wordnet')"`.
5. Start the application: `streamlit run app.py` and open `http://localhost:8501`.
6. Optionally run the tests: `python -m pytest -q`.

A.2 Using the System

1. **Sign in** with the default administrator account (`admin@resumeai.local / Admin@123`) and change the password under **Settings**.
2. On an empty database, click **Load sample data** on the Dashboard to explore the system with the demo dataset.
3. Open **Job Postings** → **New job posting**, enter the title and description (or import a TXT/PDF/DOCX file), review the detected skills and minimum experience, and save.
4. Open **Upload Resumes**, select one or more files and click **Process resumes**. Review the extracted details for each file.
5. Open **Screening & Ranking**, choose a job and click **Run screening** if needed. Adjust the threshold and click **Auto-shortlist**, or change individual statuses in the table.
6. Expand a candidate under **Why this ranking?** to see component scores and missing skills.
7. Use **Candidates** to search the pool and view recommended roles for a candidate.
8. Open **Reports**, choose a report and download it as CSV or PDF.
9. Administrators create recruiter accounts under **Settings** → **User management**.

A.3 Configuration

Table 29: Configuration options (config.py)

Setting	Default	Meaning
WEIGHT_TFIDF / WEIGHT_SKILLS / WEIGHT_EXPERIENCE	0.55 / 0.35 / 0.10	Weights of the match score components
SHORTLIST_THRESHOLD	60	Default auto-shortlist threshold (%)
MAX_FILE_MB	10	Maximum upload size
SCANNED_PDF_MIN_CHARS	50	Pages with less text are OCR-processed
TESSERACT_CMD (environment)	auto-detected	Path to tesseract executable
RSS_DB_PATH (environment)	data/resume_screening.db	Location of the SQLite database

Appendix B: Selected Source Code

The complete source code accompanies this report. Key excerpts are reproduced below.

B.1 core/matcher.py

```
"""Machine Learning Matching Module.

Score = WEIGHT_TFIDF * TF-IDF cosine similarity (rescaled)
      + WEIGHT_SKILLS * required-skill coverage
      + WEIGHT_EXPERIENCE * experience fit
The per-component scores and matched/missing skills are kept for explainability.
"""

import time

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import config
from core import nlp

# Resume/JD cosine similarity rarely exceeds ~0.6 even for strong matches (resumes contain
much
# unrelated text), so the raw value is divided by this ceiling and capped at 1.
TFIDF_CEILING = 0.6

def tfidf_similarities(job_text: str, resume_texts: list[str]) -> list[float]:
    if not resume_texts:
        return []
    vectorizer = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True, min_df=1,
token_pattern=r"(?u)\b[\w+#.]+\b")
    matrix = vectorizer.fit_transform([job_text, *resume_texts])
    return cosine_similarity(matrix[0:1], matrix[1:]).ravel().tolist()

def score_components(similarity: float, resume_skills, job_skills, experience,
min_experience) -> dict:
    job_skills, resume_skills = set(job_skills), set(resume_skills)
    tfidf_component = min(similarity / TFIDF_CEILING, 1.0)

    matched = sorted(job_skills & resume_skills)
    missing = sorted(job_skills - resume_skills)
    skill_component = len(matched) / len(job_skills) if job_skills else None

    if min_experience and min_experience > 0:
        exp_component = min((experience or 0) / min_experience, 1.0)
    else:
        exp_component = 1.0

    w_t, w_s, w_e = config.WEIGHT_TFIDF, config.WEIGHT_SKILLS, config.WEIGHT_EXPERIENCE
    if skill_component is None: # JD lists no recognisable skills: redistribute that
weight to TF-IDF
        w_t, w_s, skill_component = w_t + w_s, 0.0, 0.0
    total = w_t * tfidf_component + w_s * skill_component + w_e * exp_component

    return {
        "match_score": round(total * 100, 2),
        "tfidf_score": round(tfidf_component * 100, 2),
        "raw_cosine": round(similarity, 4),
        "skill_score": round(skill_component * 100, 2),
        "experience_score": round(exp_component * 100, 2),
        "matched_skills": matched,
```

```

        "missing_skills": missing,
    }

def rank_resumes(job: dict, resumes: list[dict]) -> list[dict]:
    """job: {job_description, skills, min_experience}; resumes: {resume_id,
    processed_text, skills, experience_years}."""
    start = time.perf_counter()
    job_processed = nlp.preprocess(job["job_description"])
    sims = tfidf_similarities(job_processed, [r["processed_text"] or "" for r in resumes])

```

B.2 core/auth.py

```

"""User Management Module: registration, login and password hashing (bcrypt)."""
import re

import bcrypt

import config
from core import database as db

EMAIL_RE = re.compile(r"^\w.+@[\w-]+(\.[\w-]+)+$")

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(password: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(password.encode("utf-8"), hashed.encode("utf-8"))
    except ValueError:
        return False

def validate_registration(name: str, email: str, password: str) -> str | None:
    """Return an error message, or None when the input is valid."""
    if not name.strip():
        return "Name is required."
    if not EMAIL_RE.match(email.strip()):
        return "Enter a valid email address."
    if len(password) < 8 or not re.search(r"\d", password) or not re.search(r"[A-Za-z]",
password):
        return "Password must be at least 8 characters and contain letters and digits."
    if db.get_user_by_email(email):
        return "An account with this email already exists."
    return None

def register(name: str, email: str, password: str, role: str = "recruiter") -> int:
    user_id = db.create_user(name.strip(), email, hash_password(password), role)
    db.log_activity(user_id, "register", f"New {role} account")
    return user_id

def login(email: str, password: str) -> dict | None:
    user = db.get_user_by_email(email)
    if user and verify_password(password, user["password"]):
        db.log_activity(user["user_id"], "login", "Successful login")
        user.pop("password", None)
        return user
    if user:
        db.log_activity(user["user_id"], "login_failed", "Wrong password")

```

```
    return None

def ensure_default_admin():
    if db.count_users() == 0:
        register("Administrator", config.DEFAULT_ADMIN_EMAIL,
config.DEFAULT_ADMIN_PASSWORD, role="admin")
```